

KERALAIM – SMART CITIZEN ASSISTANCE SYSTEM

PROJECT REPORT

Submitted by

KRISHNA DARSH E K

[EA*** 10096]**

Under the Guidance of

Dr.G.Babu

(Assistant Professor, Directorate of online Education)

in partial fulfillment for the award of the degree of

MASTER OF COMPUTER APPLICATIONS



DIRECTORATE OF ONLINE EDUCATION

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

KATTANKULATHUR- 603 203

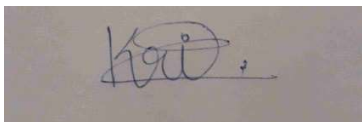
MAY-2026

**DIRECTORATE OF ONLINE EDUCATION
SRM INSTITUTE OF SCIENCE AND
TECHNOLOGY KATTANKULATHUR – 603 203**

BONAFIDE CERTIFICATE

This Project Report titled “**KERALAIM – SMART CITIZEN ASSISTANCE SYSTEM**” is the Bonafide work of **KRISHNA DARSH E K [EA2432251010096]**, completed the Project work along with the company mentor. Certified further, that to the best of my knowledge the work reported herein does not form any other Project report or dissertation based on which a degree or award was conferred on an earlier occasion on this or any other candidate

Signature of Student

A rectangular box containing a handwritten signature in blue ink. The signature appears to be 'Kd' followed by a flourish and a small mark.



AD Infocom Systems

62, Bajrang Nagar, Manewada Road, Nagpur – 440027
Contact No.: 9860455757 Email: adinfocomsystems@gmail.com
Website: www.adinfocomsystems.in



Ref. No. - ADIS/2026/194

Date - 24/05/2026

PROJECT OFFER LETTER

This is to certify that **KRISHNA DARSH E K** [EA2432251010096] has undergone Project work during the period from 24-02-2026 to 24-05-2026 under the guidance of Mr. Prabhakar Dorge.

This work has been carried out in partial fulfillment of the requirements for the award of the degree of **MASTER OF COMPUTER APPLICATIONS**.

Prabhakar Dorge
Managing Director
AD Infocom Systems



ACKNOWLEDGEMENTS

I express my humble gratitude to **Dr C. Muthamizhchelvan**, Vice-Chancellor, SRM Institute of Science and Technology, for the facilities extended for the project work and his continued support. We extend our sincere thanks to Director DOE, SRM Institute of Science and Technology, **Prof. Dr Manoranjan Pon Ram**, for his invaluable support. We want to convey our thanks to Programme Coordinator **Dr. G. Babu**, Directorate of online Education, SRM Institute of Science and Technology, for their inputs during the project reviews and support. Our inexpressible respect and thanks to my guide, **Dr G.Babu**, Assistant Professor & Programme Coordinator Directorate of online Education,, SRM Institute of Science and Technology, for providing me with an opportunity to pursue my project under his mentorship.

He provided me with the freedom and support to explore the research topics of my interest. His passion for solving problems and making a difference in the world has always been inspiring. We sincerely thank the Directorate of online Education, staff and students, SRM Institute of Science and Technology, for their help during our project. Finally, I would like to thank parents, family members, and friends for their unconditional love, constant support, and encouragement

TABLE OF CONTENTS

ABSTRACT.....	vii
INTRODUCTION.....	1
1.Problem Analysis.....	2
1.1 Initial Problem Statement.....	2
1.2 Stakeholder identification.....	2
1.3 Root cause analysis	2
1.4 Proposed System.....	3
1.5 Feasibility Study	4
2.System and Software Requirement	5
2.1 System Level Requirements.....	5
2.2 Software Requirements	6
3.Tools and Technologies.....	8
3.1 Frontend.....	8
3.2 Backend.....	9
3.3 Testing Framework	11
3.4 AI API.....	12
3.5 Other Services	13
3.6 AI Models	14
4. Design.....	15
4.1 DATA FLOW DIAGRAM.....	15
4.2 ER DIAGRAM	18
4.3 DATABASE DESIGN	19
4.4 INPUT DESIGN	23
4.5 OUTPUT DESIGN.....	23
5.Module Description	24
6. Implementation	28
6.1 Application Overview.....	28
6.2 Server-Side Implementation	28
6.3 Mobile application implementation	37

7. Testing.....	54
7.1 Testing methodology	54
7.2 TEST CASE AND OUTPUTS	56
8.CONCLUSION.....	59
9.APPENDICES	60
10. REFERENCES	67

ABSTRACT

KeralAim is smart platform designed to streamline citizen access to Kerala government schemes and public information driven through AI and ML. It bridges the gap between citizens and administration with features like intelligent scheme recommendations, automated scheme eligibility analysis based on demographic inputs, chatbot functionality, complaint classification, nearby government services and predictive analytics of disaster and alert mechanism . The system integrates advanced technologies including machine learning classifiers, recommendation engines, and geospatial mapping. Key modules handle grievance categorization via image classification, personalized schemes and smart locators for nearby facilities like hospitals and offices. Built on a modular microservices architecture with React as frontend, Node.js as backend, MongoDB as database, and AI services via AI APIs. This scalable system boosts AI-driven e-governance by smartly combining data analysis with everyday government service needs in developing areas.

INTRODUCTION

The increasing complexity of knowing government services and public welfare schemes remains a significant challenge for citizens, especially in regions with diverse socioeconomic backgrounds such as Kerala. Many individuals are often unaware of the schemes they are eligible for, face difficulties in navigating multiple platforms, unaware of eligible government exams or lack a centralized system to address civic issues efficiently. This results in underutilization of welfare programs and delays in resolving public concerns.

The “**Keralaim**” project aims to address these challenges by developing a unified digital platform that integrates government schemes, complaint management, and disaster alert systems into a single, user-friendly application. The platform leverages modern technologies to provide personalized recommendations, enabling users to discover relevant schemes based on their profile, location, and needs.

“**Keralaim**” focuses on enhancing accessibility and transparency by allowing users to easily explore schemes, submit complaints with supporting media, and track their status in real time. Additionally, the integration of disaster alert modules ensures that users receive timely updates on critical situations such as floods, heavy rainfall, and other emergencies, thereby improving public safety and preparedness.

By bridging the gap between citizens and government services, “**Keralaim**” contributes to efficient governance and digital transformation. The platform is designed to cater to a wide range of users, including students seeking financial assistance, workers looking for welfare schemes, and citizens reporting local issues in their communities. It ensures that users are not only informed but also empowered to take action through a streamlined and responsive system.

“**Keralaim**” reflects the growing importance of smart governance and citizen centric applications in the digital era. By combining scalable backend systems, mobile first design, and real time data integration, the project provides a practical implementation of modern software engineering principles while addressing real world social needs. It envisions a future where technology simplifies governance, enhances civic engagement, and contributes to building a more connected and responsive society.

1.Problem Analysis

This section covers the analysis of the problem in existing system, identifying the stakeholders in the system and understand the core issue

1.1 Initial Problem Statement

The existing apps don't have AI features integrated to their application. There are separate applications which provide schema recommendation, disaster alerts, complaint registration but there is not unified system which provide all these requirements in one application. Regarding the complaint registration feature, the current one is manual, slow and scattered process of submitting complaint to represent authorities through application

Scheme suggestion feature is also not personalized. The existing system show all schema to all users who may or may not be eligible for that scheme

1.2 Stakeholder identification

- **Citizens (Users)** – Individuals who use the platform to explore schemes, submit complaints, and receive alerts
- **Government Authorities / Admin** – Manage schemes, monitor complaints, update statuses, and view analytics
- **External Systems** – APIs providing weather data , AI API and disaster alerts (e.g., weather and AI integration)

1.3 Root cause analysis

Using a cause-and-effect approach, the following issues are identified:

- **Lack of awareness:** Citizens are unaware of eligible schemes due to scattered information across multiple portals
- **No personalization:** Existing systems do not recommend schemes based on user profile or needs

Unaware of job opportunity : There is no platform to know which exam's we are eligible

- **Fragmented platforms:** Separate systems for schemes, complaints, and alerts lead to poor user experience
- **Inefficient complaint handling:** Lack of real-time tracking and transparency in

complaint resolution

- **Delayed disaster communication:** Alerts are not centralized or timely, affecting public safety
- **Poor accessibility:** Complex interfaces and lack of mobile-friendly design reduce usability
- **Manual processes:** Many services still rely on offline/manual workflows, causing delays

1.4 Proposed System

The proposed system, “Keralaim”, overcomes the limitations of existing government service platforms by introducing an AI-powered citizen assistance platform that provides personalized access to government schemes, complaint management, disaster alerts, and public service information within a single unified application. The system is designed to simplify interactions between citizens and government services while improving accessibility, transparency, and responsiveness.

The platform allows users to securely register and log in, after which they complete an intelligent onboarding questionnaire to capture important eligibility-related details such as age, occupation, education, district, income category, and other relevant information. Based on this data, the AI-powered recommendation engine analyzes government scheme eligibility criteria and suggests the most relevant schemes for each user. The recommendation system dynamically adapts as the user updates their profile or answers additional AI-generated questions.

The system also includes a smart complaint registration module where users can upload complaint images, add descriptions, and provide location details. Artificial Intelligence is used to automatically classify complaints, generate structured descriptions, identify urgency levels, and route complaints to the appropriate government department. Users can track the complaint lifecycle through statuses such as submitted, in progress, and resolved, improving transparency and trust.

In addition, KeralaIM integrates a nearby government service locator that helps users discover hospitals, police stations, Akshaya centers, panchayat offices, and other public institutions near their current location. The platform also provides a disaster and weather alert module that analyzes weather data and generates localized alerts for heavy rainfall, flood risks, and emergency situations. Push notifications are used to inform users about critical updates such as disaster warnings or complaint status changes.

The platform further includes an AI chatbot assistant. Citizens can ask questions related to schemes, services, complaints, and government procedures, and the chatbot responds with structured guidance and recommendations. An admin dashboard is also included for monitoring complaints, tracking analytics, managing notifications, and analyzing application usage metrics.

1.5 Feasibility Study

1.5.1 Technical Feasibility

The technical feasibility of the KeralaIM system is high because it is built using modern and reliable technologies such as React Native with Expo for the mobile frontend, Node.js and Express.js for the backend, and MongoDB as the database. These technologies are open-source, scalable, and widely supported by the developer community.

The platform integrates several external technologies and APIs including Cloudinary for image storage, Expo Notifications for push notifications, Open-Meteo APIs for weather monitoring, and AI APIs such as Gemini/OpenRouter for chatbot assistance, complaint classification, and eligibility analysis. The use of RESTful APIs, JWT authentication, and modular architecture ensures that the application remains maintainable, secure, and scalable.

The AI-based recommendation engine and dynamic questionnaire system further enhance the technical robustness of the project by enabling personalized government scheme recommendations based on real-time eligibility analysis.

1.5.2 Economic Feasibility

Economically, the system is feasible because it primarily relies on free and open-source technologies. Most of the services used in the project, including MongoDB Atlas, Expo, Open-Meteo APIs, and Cloudinary free tiers, significantly reduce infrastructure and operational costs.

The application can be deployed using low-cost or free-tier cloud platforms such as Render, Railway, or Vercel, making it suitable for academic projects, startups, and government prototypes. The AI integrations also use free or limited-cost APIs such as Gemini Flash models and OpenRouter free-tier services, minimizing development expenses while still providing intelligent functionality.

As a result, the project can be developed and maintained with minimal financial investment while still delivering advanced features and scalability.

1.5.3 Operational Feasibility

From an operational perspective, KeralaIM is highly feasible because the system is designed with a user-friendly and intuitive interface that can be easily used by citizens from different educational and technical backgrounds.

Users can smoothly navigate through registration, scheme recommendations, complaint submission, disaster alerts, and chatbot assistance without requiring technical expertise. Features such as multilingual chatbot support, simple forms, location-based services, and real-time notifications improve accessibility and usability.

The admin dashboard simplifies monitoring and management tasks for administrators by providing centralized access to complaint tracking, analytics, and notification management. The automation of complaint classification, recommendation generation, and alert systems also reduces manual workload and improves operational efficiency.

Overall, the platform provides a practical and efficient solution for improving citizen access to government services and digital governance systems.

2.System and Software Requirement

Requirements are typically specified at two levels: system and software. System requirements define the overall functionality and constraints, while software requirements provide more detail on the software component

2.1 System Level Requirements

2.1.1 Functional Requirements

- **SR1 — Authentication:** Users can register/login using email/password and secure authentication
- **SR2 — User Profile:** Users can manage profile details (age, location, preferences)
- **SR3 — Scheme Discovery:** Users can browse and search government schemes
- **SR4 — Personalized Recommendations:** System suggests schemes based on user profile and eligibility
- **SR5 — Complaint System:** Users can submit complaints with image/video proof
- **SR6 — Complaint Tracking:** Users can track complaint status in real-time
- **SR7 — Disaster Alerts:** System provides real-time alerts (rain, flood, etc.) based on location
- **SR8 — Admin Dashboard:** Admin can manage schemes, complaints, and alerts
- **SR9 — Notifications:** Users receive updates on complaints and alerts
- **SR10 — Data Synchronization:** System integrates with external APIs for schemes and ale

2.1.2 Non-Functional Requirements

- **SR11 (Performance):** System should handle 1000+ concurrent users with minimal latency (<2 seconds response time)
- **SR12 (Security):** Data must be secured using encryption, JWT authentication, and secure storage
- **SR13 (Availability):** System should maintain 99.9% uptime
- **SR14 (Usability):** Mobile-first, intuitive UI accessible to all user groups
- **SR15 (Scalability):** System should scale horizontally to support increasing users
- **SR16 (Reliability):** System should handle failures gracefully and ensure data consistency
- **SR17 (Maintainability):** Modular architecture for easy updates and debugging

2.2 Software Requirements

2.2.1 Functional Requirements

- POST /auth/register → body {name,email,password} → returns JWT (201)
- POST /auth/login → body {email,password} → returns JWT (200)
- GET /me → returns user profile (200)
- PUT /me → update profile (200)
- GET /schemes → fetch all schemes (200)
- GET /schemes/:slug → fetch scheme details (200)
- GET /schemes/recommendor → personalized schemes (200)
- POST /complaints → submit complaint (image + location) (201)
- GET /complaints → get user complaints (200)
- GET /complaints/me → get complaint details (200)
- GET /alerts → fetch latest alerts (200)
- GET /alerts/:district → district-specific alerts
- POST /schemes → add scheme (ADMIN)
- PUT /schemes/:id → update scheme
- DELETE /schemes/:id → delete scheme
- GET /admin/analytics → usage metrics

2.2.2 Non-Functional Requirements

- **Database:** Use NoSQL database (MongoDB) for scalable data storage
- **API:** RESTful API architecture for frontend-backend communication
- **Framework:**
 - **Frontend:** React Native (Expo)
 - **Backend:** Node.js (Express)
- **Security:**
 - JWT-based authentication
 - Secure storage (SecureStore)
 - HTTPS encryption
- **Logging:**
 - Log errors and user actions for debugging and monitoring
- **Performance Optimization:**
 - Caching frequently accessed data
 - Optimized queries
- **Compatibility:**
 - Supports Android and iOS devices

3.Tools and Technologies

Tools and technologies used to develop an application must be cleverly chosen in order to optimize the performance and generate a great user experience for the final end users using our product. There are plenty of tools in current market to choose from for developing a website. There a lot of frameworks, libraries and packages which will help us to develop website fast and easily. The selection of these tools was based on their performance, ecosystem maturity, ease of integration, and suitability for the project's functional goals. So, the tools and technologies I used in my mini project are as follows:

3.1 Frontend

A) React Native

React Native is an open-source JavaScript framework designed for building native mobile applications for iOS and Android using React principles. It enables developers to create encapsulated, reusable native UI components that manage their own state, delivering high-performance mobile apps with a single codebase across platforms.

React Native simplifies cross-platform mobile development through its component-based architecture and declarative programming. It uses JSX syntax blending JavaScript with native component markup, and leverages the native rendering pipeline for true native performance and feel.

B) Frontend UI Libraries

i) Tailwind CSS : Tailwind CSS is a modern, utility-first CSS framework that allows developers to rapidly build custom user interfaces by applying predefined CSS classes directly within HTML. Unlike traditional CSS frameworks that offer pre-styled components, Tailwind provides small, atomic utility classes like p-4 (padding), text-center (text alignment), and bg-blue-500 (background color) that can be combined to create any design. It emphasizes speed, flexibility, and maintainability, making responsive design and

customization straightforward without writing custom styles, It is a popular choice for web development projects including mini projects.

ii) React query: React Query is a powerful data-fetching and state management library for React applications that abstracts and simplifies server state handling. It provides an intuitive API to fetch, cache, synchronize, and update data seamlessly, without boilerplate code. Key features include intelligent caching to reduce redundant network requests, background re-fetching to keep data up-to-date without disrupting the user experience, automatic retries on failed requests, and query invalidation to refresh stale data. React Query enhances performance, reduces manual coding, and offers optimistic updates for responsive UI. Its flexibility and integration with React ecosystem make it a preferred choice for managing asynchronous data in modern React projects, greatly improving developer productivity and user experience

3.2 Backend

A)Node.js

Node.js is an open-source, cross-platform JavaScript runtime environment that allows developers to execute JavaScript code outside of a web browser, primarily on the server side. It is built on Chrome's V8 JavaScript engine, enabling fast and efficient execution of code. Node.js is widely used for building scalable and high-performance web applications, APIs, real-time services, and microservices due to its event-driven, non-blocking I/O model. Node.js is well-suited for mini projects due to its simplicity, fast setup, and vast library ecosystem. It allows full-stack JavaScript development, keeping the stack consistent by using JavaScript on both client and server sides. Its asynchronous nature ensures responsive applications with good performance

B) MongoDB

MongoDB is a popular NoSQL, document-oriented database designed to store, manage, and retrieve large volumes of unstructured and semi-structured data efficiently. Unlike traditional relational databases, MongoDB uses a flexible schema model that stores data in JSON-like BSON (Binary JSON) documents, making it ideal for projects requiring agility and scalability. ideal for evolving data models (nested documents, arrays, AI logs).

why chosen : The Db is flexible with data storing and can store somany nested values without need of other table

C) Mongoose

Mongoose is an Object Document Mapper (ODM) for MongoDB and Node.js that provides a schema-based solution to model application data. It abstracts MongoDB interactions by allowing developers to define schemas that enforce the shape, data types, and validation rules for documents in a MongoDB collection. This abstraction simplifies working with MongoDB by providing built-in validation, middleware (hooks), and easy CRUD operations.

D) Express.js

Express.js is a minimal and flexible Node.js web application framework that simplifies building robust and scalable web applications and APIs by providing essential features such as routing, middleware support, and templating. It streamlines handling HTTP requests and responses, enabling developers to easily build RESTful services or dynamic web pages. Express's unopinionated nature allows developers to structure their projects as they prefer, while its middleware architecture supports adding functionalities like authentication, logging, and error handling.

why chosen : easy api and large ecosystem

.

3.3 Testing Framework

A) Jest

Jest is a widely used, full-featured JavaScript testing framework developed by Meta, built for reliability and simplicity across web and Node.js applications. It offers a batteries-included experience with zero-configuration setup for most projects, powerful mocking and spying utilities, snapshot testing, and comprehensive assertion APIs such as test, describe, and expect. Jest runs tests in parallel worker processes for speed, supports code coverage reporting via Istanbul, and integrates smoothly with Babel and TypeScript through transformers. As a popular choice for React and Node ecosystems, Jest works well in MERN stack workflows, monorepos, and CI pipelines, and is easily used within development environments like VS Code alongside Tailwind CSS and React Native projects, making it a robust option for full-stack developers seeking stability and a rich plugin ecosystem.

why chosen : large ecosystem and easy syntax

B) TunderClient

Thunder Client is a lightweight REST API client extension for VS Code (and JetBrains IDEs), hand-crafted for simplicity, clean design, and local storage to enable fast API testing without context switching from your code editor. It supports collections, environment variables, scriptless GUI-based response testing, Git sync for team collaboration, GraphQL queries, and an advanced CLI for CI/CD integration, making it ideal for MERN stack developers like you working with Node.js/Express APIs alongside Vitest testing workflows. Launched in 2021, it prioritizes privacy with 100% local data storage and has gained millions of downloads as a Postman alternative focused on speed and offline capability

why chosen : Intergrated in VScode as an extension , lightweight compared to Postman

3.4 AI API

1) OpenRouter AI API

OpenRouter AI API is a unified, OpenAI-compatible gateway that provides developers with seamless access to over 400 large language models (LLMs) from more than 50 provider including OpenAI, Anthropic, Google, Meta, Mistral, and others via a single endpoint (<https://openrouter.ai/api/v1>), eliminating vendor lock-in and simplifying integration, automatic fallbacks for reliability, provider normalization, cost controls, and support for multi-modal inputs (text/images), making it perfect for the FastAPI/Node.js backends in MERN/SaaS projects to dynamically select models based on needs like reasoning or speed. Authentication uses Bearer tokens with free tiers, pay-as-you-go pricing, and SDK compatibility .

Usage : Image classification for complaints

2) GROQ_API

Groq API key gives developers access to Groq's fast AI inference platform, which is built for low-latency execution of large language models through a simple authenticated API. It is commonly used to power chatbots, assistants, and backend AI features in Node.js, FastAPI, and MERN applications where speed matters, and it supports integration with OpenAI-style SDK patterns in many workflows. After creating a Groq account, you generate an API key from the dashboard and use it as a Bearer token in your server-side requests, which makes it suitable for secure deployment in SaaS projects, developer tools, and real-time applications

Usage : Parsing eligibility , Ai chatboat , Ai description generation for complaints

3.5 Other Services

1) FCM

FCM API key is used to connect your application with Firebase Cloud Messaging, enabling push notifications to Android, iOS, and web users through a secure backend setup. It is commonly used in FastAPI, Node.js, and MERN projects to send real-time alerts, updates, and engagement messages without exposing sensitive notification logic on the frontend.

Usage : Push Alerts

2) Cloudinary

Cloudinary API key is used to authenticate your application with Cloudinary's media platform for uploading, storing, transforming, and delivering images and videos efficiently. It is especially useful in FastAPI, Node.js, and MERN applications where media handling, optimization, and CDN-based delivery are needed for scalable web and SaaS projects

Usage : Store the complaints photo in cloud , supports multiple uploads

3) OpenstreetMAP API

OpenStreetMap API is a mapping and geodata service used to access and edit map data, making it useful for location-based features in web and mobile applications. In practice, developers often pair OSM data with services like Nominatim for geocoding, while following usage policies such as valid user identification, rate limits, caching, and proper attribution to ensure reliable and compliant integration.

Usage : Fetch Nearby Government services through tags.

3.6 AI Models

1) google/gemini-2.0-flash

Google Gemini 2.0 Flash is a high-performance multimodal AI model from Google designed for fast responses, strong reasoning, and everyday developer use cases. It supports text, image, audio, and video inputs, provides a 1 million token context window, and is available through the Gemini API in Google AI Studio and Vertex AI, making it ideal for modern FastAPI, Node.js, and MERN applications that need scalable AI feature

Usage: Image Classification for routing complaints to different department

2) llama-3.1-8b-instant

llama-3.1-8b-instant is a high-speed, instruction-tuned language model provided by Groq for real-time AI applications. It offers a 128K token context window, strong tool-use and JSON output support, and very low-latency inference, making it a practical choice for FastAPI, Node.js, and MERN projects that need fast, cost-effective text generation and conversational features.

Usage: Ai complaint description generator , smart eligibility question generator , parsing eligibility of schemes

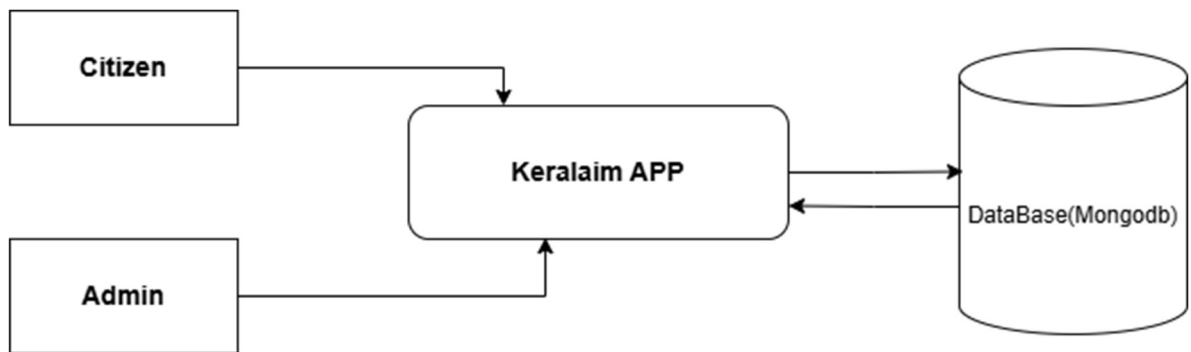
4. Design

The major contributor for a successful project is the design of the project. Every project building start with a clear design of the system. The design should include how the data flow through the system, usecases of the system and the relationship between entities in the system. In this project I use DFD, ER, usecase , sequence and activity to illustrate the overall system and also added the database design to show how and what values are stored and their linking using foreign key

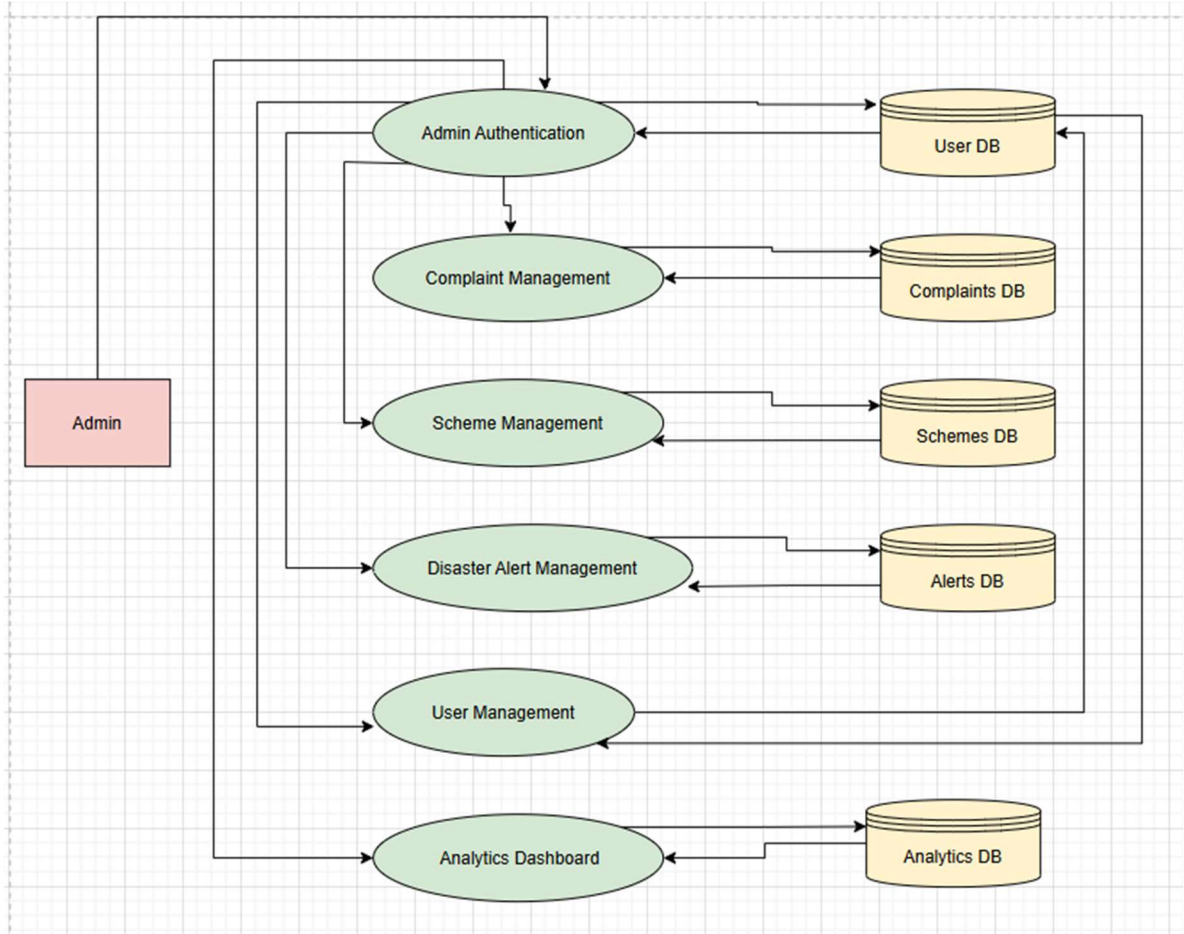
4.1 DATA FLOW DIAGRAM

A data flow diagram (DFD) maps out the flow of information for any process or system. It uses defined symbols like rectangles, circles and arrows, plus short text labels, to show data inputs, outputs, storage points and the routes between each destination. Data flowcharts can range from simple, even hand-drawn process overviews, to in-depth, multi-level DFDs that dig progressively deeper into how the data is handled

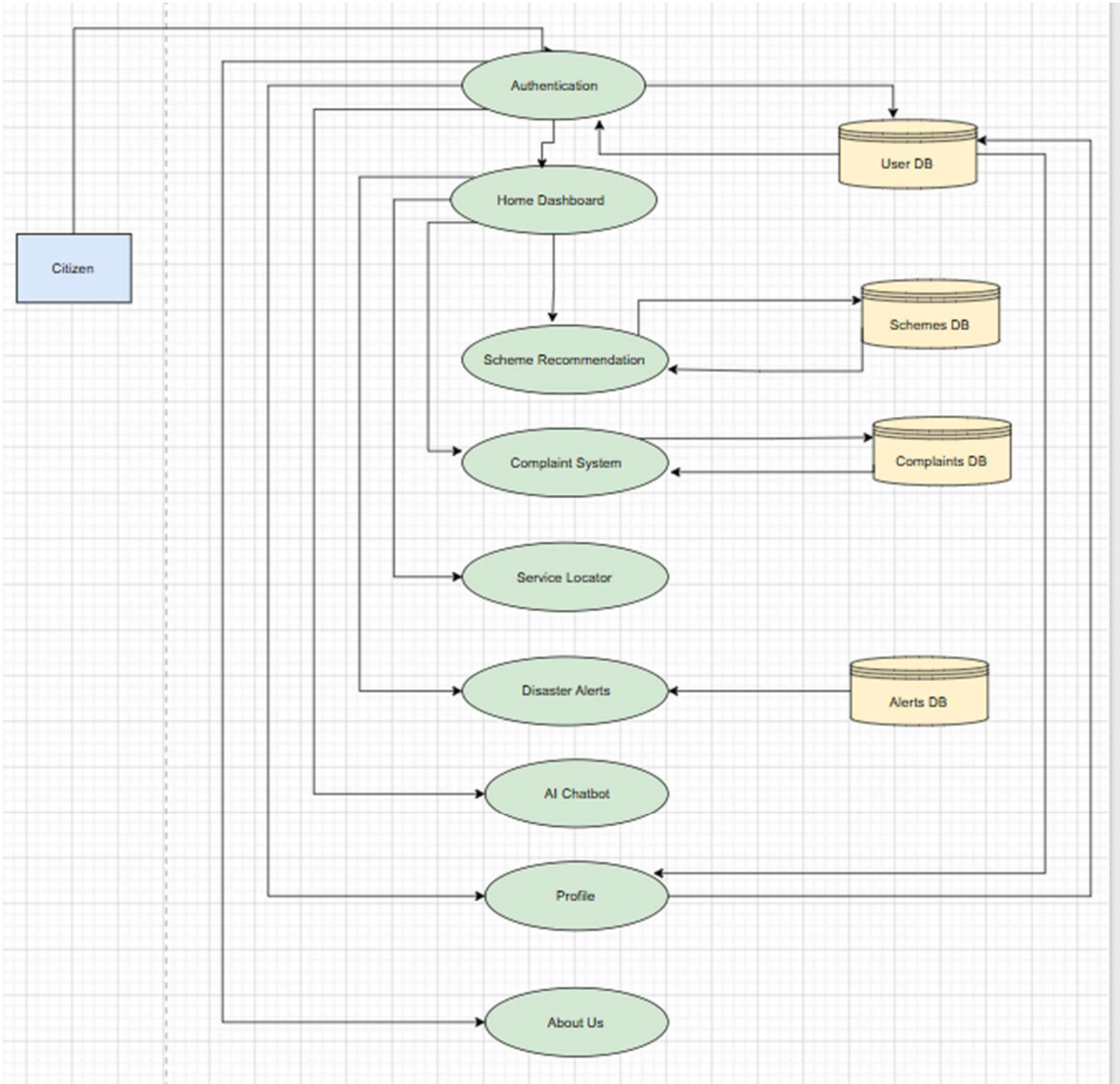
LEVEL 0 DFD



Level 1 DFD: LEVEL 1.1 DFD (Admin)

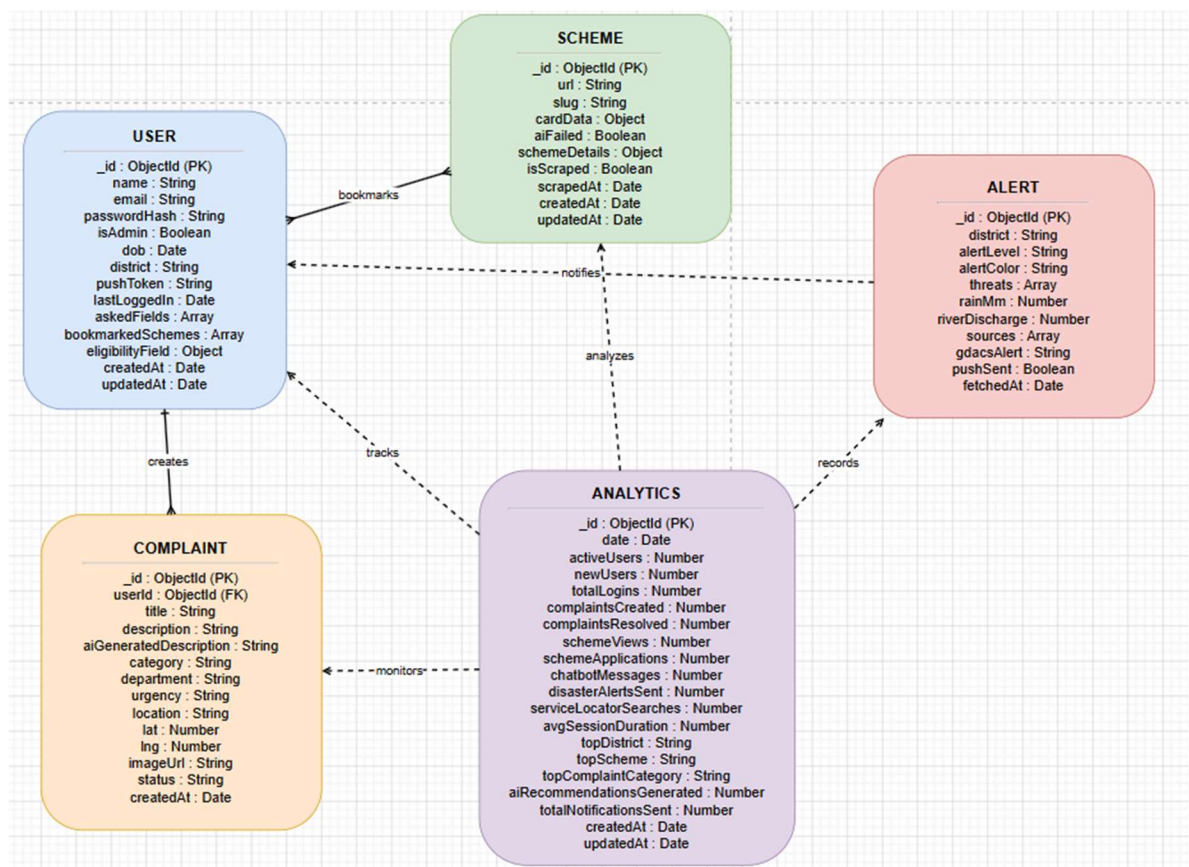


LEVEL 1.2 DFD (Learner)



4.2 ER DIAGRAM

An Entity Relationship (ER) Diagram is a type of flowchart that illustrates how “entities” such as people, objects or concepts relate to each other within a system. ER Diagrams are most often used to design or debug relational databases in the fields of software engineering, business information systems, education and research. Also known as ERDs or ER Models. They mirror grammatical structure, with entities as nouns and relationships as verbs



4.3 DATABASE DESIGN

Database design is one of the most important aspects of programming. Successful implementation of any system with Data Storage will always need proper database design. To create a great database design, you will not only have to master the Database technology but will have to master the database design process and normalization.

Table Structure

4.3.1 USER

Data Type	Attribute	Description
ObjectId	<u>id</u>	PK
String	name	Name of the user
String	email	Email of the user
String	passwordHash	Hashed password
Boolean	isAdmin	Admin or not
Date	dob	Date of birth
String	district	District of the user
String	pushToken	Token for push notifications
Date	lastLoggedIn	When the user last logged in
Array of Strings	askedFields	Stores already asked eligibility questions
Array of ObjectId	bookmarkedSchemes	Saved/bookmarked schemes
Date	createdAt	When created
Date	updatedAt	When the profile was updated
Object Type	eligibilityField	Stores user eligibility information used for scheme recommendations

4.3.2 Scheme

Data Type	Attribute	Description
ObjectId	_id	PK
String	url	Original scheme source URL
String	slug	Unique slug identifier for the scheme
Object	cardData	Stores summarized card information of the scheme
Boolean	aiFailed	Indicates whether AI parsing failed
Object	schemeDetails	Stores complete scheme details
Object	schemeDetails.details	Detailed description of the scheme
Array of Object	schemeDetails.details.structured	Structured content blocks for scheme details
String	schemeDetails.details.plainText	Plain text version of scheme details
Object	schemeDetails.benefits	Benefits provided by the scheme
Array of Object	schemeDetails.benefits.structured	Structured content blocks for benefits
String	schemeDetails.benefits.plainText	Plain text version of benefits
Object	schemeDetails.eligibility	Eligibility criteria of the scheme
Array of Object	schemeDetails.eligibility.structured	Structured eligibility content
String	schemeDetails.eligibility.plainText	Plain text eligibility details
Mixed	schemeDetails.eligibility.parsed	AI parsed eligibility fields
Object	schemeDetails.exclusions	Exclusion criteria of the scheme
Array of Object	schemeDetails.exclusions.structured	Structured exclusion content
String	schemeDetails.exclusions.plainText	Plain text exclusion details
Object	schemeDetails.documentsRequired	Required documents for application
Array of Object	schemeDetails.documentsRequired.structured	Structured required document content
String	schemeDetails.documentsRequired.plainText	Plain text required document details
Object	schemeDetails.applicationProcess	Application process details

Data Type	Attribute	Description
Object	schemeDetails.applicationProcess.online	Online application process
Array of Object	schemeDetails.applicationProcess.online.structured	Structured online application steps
String	schemeDetails.applicationProcess.online.plainText	Plain text online application steps
Object	schemeDetails.applicationProcess.offline	Offline application process
Array of Object	schemeDetails.applicationProcess.offline.structured	Structured offline application steps
String	schemeDetails.applicationProcess.offline.plainText	Plain text offline application steps
Boolean	isScraped	Indicates whether scheme data was scraped
Date	scrapedAt	Date and time when scheme was scraped
Date	createdAt	When the scheme document was created
Date	updatedAt	When the scheme document was last updated

4.3.3 Analytics

Data Type	Attribute	Description
ObjectId	id	Unique MongoDB document identifier
Date	date	Date for which analytics data is stored
Number	activeUsers	Total daily active users
Number	newUsers	Number of newly registered users
Number	totalLogins	Total login count for the day
Number	complaintsCreated	Total complaints submitted
Number	complaintsResolved	Total resolved complaints
Number	complaintsInProgress	Total complaints currently in progress
Number	schemeViews	Number of government scheme detail views
Number	schemeApplications	Number of scheme application redirects/clicks
Number	chatbotMessages	Total AI chatbot messages exchanged
Number	disasterAlertsSent	Total disaster alert notifications sent
Number	emergencyReports	Number of emergency/disaster reports created

Data Type	Attribute	Description
Number	serviceLocatorSearches	Number of nearby service searches performed
Number	avgSessionDuration	Average user session duration in minutes
String	topDistrict	District with highest user activity
String	topScheme	Most viewed government scheme
String	topComplaintCategory	Most reported complaint category
Number	aiRecommendationsGenerated	Number of AI-generated recommendations
Number	totalNotificationsSent	Total push notifications sent
Date	createdAt	Document creation timestamp
Date	updatedAt	Document last updated timestamp

4.3.4 Complaint

Data Type	Attribute	Description
ObjectId	<u>id</u>	PK
ObjectID	userId	FK
String	title	Title of complaint
String	description	Description of complaint
String	aiGeneratedDescription	Description generated by AI
String	category	water, waste, etc
String	department	PWD, Water Authority, etc
String	urgency	Type of urgency
String	location	Location details
Number	lat	Latitude
Number	lng	Longitude
String	imageUrl	Cloud URL of image
String	status	submitted, inProgress, resolved
Date	createdAt	When created

4.3.5 Alert

Data Type	Attribute	Description
ObjectId	<u>id</u>	PK
String	district	Which district
String	alertLevel	It can be green, orange, red
String	alertColor	The color code of alert
Array of String	threats	What all threats
Number	rainMm	The depth of water accumulated on the surface
Number	riverDischarge	Rate of river discharge
Array of String	sources	Which source the data is from
String	gdacsAlert	The GDACS alert description
Boolean	pushSent	Whether the push notification was sent or not
Date	fetchesAt	The date and time at which the data was fetched from API

4.4 INPUT DESIGN

Design input is the starting point for product design. The requirements which form the design input establish a basis for performing subsequent design tasks and validating the design. Therefore, development of a solid foundation of requirements is the single most important design control activity.

4.5 OUTPUT DESIGN

Design outputs describe all of the ingredients that go into your device. These could include drawings, components, materials, parts, pieces, specifications, manufacturing instructions, and inspection procedures

5.Module Description

In a modularized project, the work is divided into smaller, self-contained units of functionality called modules. A module description focuses on the internal workings and specific purpose of each of these unit. “*Keralaim*” project is divided into 9 modules with each module have specific functionality. The 8 modules are:

5.1 User Management Module

- ❖ **Purpose:** To manage user authentication and profile information securely.
- ❖ **Input and Output:**
 - o Input: User credentials (email, password), profile details (name, age, location)
 - o Output: Authenticated session (JWT), user profile data
- ❖ **Functional Requirements:**
 - o Register new users
 - o Authenticate users (login/logout)
 - o Update user profile details
- ❖ **Non-functional Requirements:**
 - o Authentication response time < 2 seconds
 - o Secure password storage (hashed & salted)

5.2 Scheme Recommendation Module

- ❖ **Purpose:** To provide personalized government scheme recommendations based on user profile.
- ❖ **Input and Output:**
 - o Input: User details (age, location, category)
 - o Output: List of relevant schemes with details
- ❖ **Functional Requirements:**
 - o Fetch schemes from database
 - o Filter schemes based on eligibility

- o View detailed scheme information
- o Redirect to official application website
- ❖ **Non-functional Requirements:**
 - o Response time < 2 seconds
 - o High accuracy in recommendations

5.3 Smart Complaint Registration Module

- ❖ **Purpose:** To allow users to register and track complaints efficiently with smart classification and routing.
- ❖ **Input and Output:**
 - o Input: Image/video, description, location
 - o Output: Complaint ID, status updates imageUrl
- ❖ **Functional Requirements:**
 - o Upload complaint with media (Cloudinary)
 - o Classify complaint type (AI)
 - o Route complaint to relevant authority
 - o Track complaint status
- ❖ **Non-functional Requirements:**
 - o File upload should complete within 5 seconds
 - o System should handle multiple concurrent uploads

5.4 Nearby Government Locator Module

- ❖ **Purpose:** To help users locate nearby government services based on their location.
- ❖ **Input and Output:**
 - o Input: User location (GPS)
 - o Output: List of nearby facilities (hospital, police station, etc.)
- ❖ **Functional Requirements:**
 - o Fetch nearby locations
 - o Display location details using OpenstreetMAP API
 - o Open location in maps

5.5 Disaster Alert and Weather Module

- ❖ **Purpose:** To provide real-time weather updates and disaster alerts for public safety.
- ❖ **Input and Output:**
 - o Input: Location, external API data
 - o Output: Weather info, alert notifications
- ❖ **Functional Requirements:**
 - o Fetch weather data
 - o Display rain alerts (green/yellow/orange/red)
 - o Send emergency notifications
- ❖ **Non-functional Requirements:**
 - o Real-time updates with minimal delay
 - o High reliability during critical events
- ❖ **Dependencies:**
 - o Weather APIs (e.g.,)
 - o Notification services

5.6 AI Chatbot Assistance Module

- ❖ **Purpose:** To provide intelligent assistance for user queries related to schemes and services.
- ❖ **Input and Output:**
 - o Input: User queries (text/voice)
 - o Output: Answers, recommendations, to-do steps
- ❖ **Functional Requirements:**
 - o Answer queries using AI
 - o Support multiple

5.7 Admin Module

- ❖ **Purpose:** To enable administrators to monitor, manage, and control the overall system operations including complaints, schemes, and analytics.

❖ **Input and Output:**

- o Input: Complaint data, user activity data, scheme updates
- o Output: Updated complaint statuses, analytics dashboards, system reports

❖ **Functional Requirements:**

- o View and manage all user complaints
- o Update complaint status (e.g., Submitted → In Progress → Resolved)
- o Manage schemes (add/edit/delete)
- o Monitor user activity and application usage
- o View analytics (number of users, complaints, active sessions)

❖ **Non-functional Requirements:**

- o Dashboard should load within 2 seconds
- o Secure access with role-based authentication (admin-only)
- o High reliability for real-time monitoring

5.8 AI / ML Module

- ❖ **Purpose:** To enhance system intelligence by enabling smart recommendations, complaint classification, and chatbot assistance using AI/ML techniques.

❖ **Input and Output:**

- o Input: User profile data, complaint text/images, user queries
- o Output: Recommended schemes/jobs, classified complaints, chatbot responses

6. Implementation

This section is structured in three main parts for a comprehensive system overview. The first part details the application's key features and user functionalities. The second covers server-side architecture, including design, technologies, request handling, and data processing. The third focuses on the front-end mobile app, explaining UI development, user interactions, and seamless mobile experience delivery. This organization clearly separates conceptual overview, backend logic, and frontend implementation for maximum clarity

6.1 Application Overview

The Keralain Smart Citizen Assistance System is a mobile-based platform designed to assist citizen in identifying their appropriate schemes, search for nearby government services, post complaint and receive disaster alert in case of emergence. The application includes two primary roles: Admin(Government) and User (Citizen).

For Users, the system provides intelligent recommendations of schemes based on user data. Ai chat boat to ask any doubt regarding government services and schemes, disaster alert ,grievance system .For Admins, the platform provides analytics and the complaint records The overall aim is to bridge gap between citizen and government services and schemes.

6.2 Server-Side Implementation

6.2.1 API Planning

The Application Programming Interface (API) serves as the communication layer between the front-end client (React-based web interface) and the server-side backend (Node.js + Express).

It facilitates data exchange between users, the AI model, and the database. Each API endpoint has a specific path, HTTP method, required input, and a well-defined purpose.

The table below outlines the major API routes designed and implemented in this project:

Table 1. API Paths

Endpoint	Method	Input	Output	Description
/api/auth/register	POST	name, email, password	userid , token, isAdmin	Registers a new user and creates an account in the database.
/api/auth/login	POST	email, password	userid , token, isAdmin	Authenticates user and returns JWT token for secure session handling.

/api/users/me	GET	Authorization (JWT)	Name,district, religion, category, annual income	Retrieves logged-in user profile details
/api/users/me	PUT	Authorization (JWT), user data	updated profile	Updates user profile (age, occupation, income, etc.).
/api/schemes/:slug	GET	Authorization (JWT)	Scheme details eligibility documents req	Fetches detailed information of a specific scheme.
/api/schemes/:slug/question	GET	Scheme eligibility details	Question related to that schemes	Generate question for a scheme to know the user eligibility
/api/schemes/recommender	GET	Userid,page	Displays Recommended schemes based on eligibility profile	Uses rule-based + AI logic to recommend relevant schemes.
/api/complaints/me	GET	Authorization (JWT)	title, description status	Retrieves all complaints submitted by the logged-in user.
/api/complaints	POST	Authorization (JWT), image, location, description	Success or not message	Submits a new complaint with image upload.
/api/service-locator/nearby	GET	latitude, longitude	name,type distance	Fetches nearby government services (hospital, police, etc.).
/api/ai/chat	POST	message	reply	AI chatbot responds to user queries about schemes/services.
/api/alerts	GET	District,level,limit	Combined alert from diff api	Used to get alerts of all districts,single district or alert type

6.2.2 Server Implementation

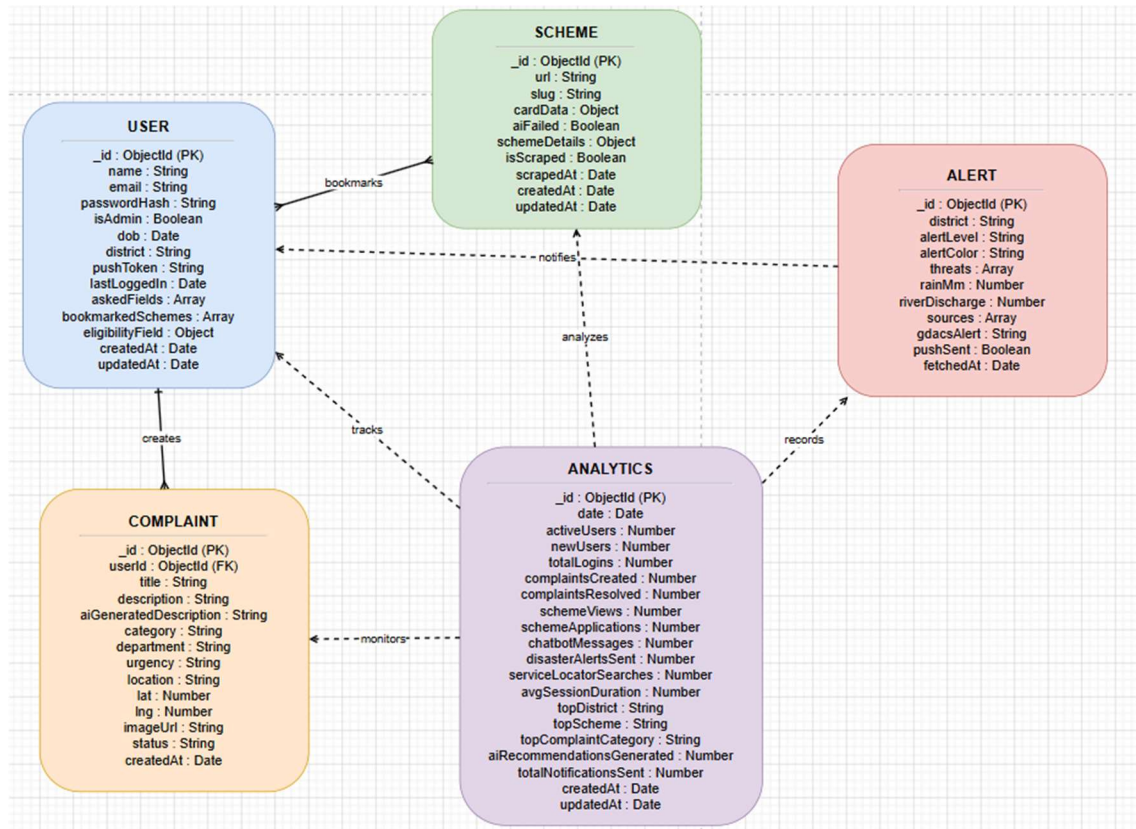
Before implementing the server, a database is required to store all the application data. In this project, MongoDB was chosen as the database system due to its flexible schema, high scalability, and suitability for AI-driven applications where the data structure can evolve dynamically.

Database Collections

The database for this project contains **five main collections**:

1. **Users Collection** – Stores all user-related information including login credentials, userDetails, isAdmin and lastLogin.
 - Fields: _id, name, email, passwordHash, userDetails, isAdmin
2. **Schemes Collection** – Contains details of all Schemes scrapped from MyScehemes
 - Fields: _id, title, SchemeDetails[], isScrapped, slug, url
3. **Complaint Collection** – Stores the user submitted complaint and status of the complaint .
 - Fields: userId , description, title, location, status, createdAt
4. **Alerts Collection**– Used for storing alerts gerated from different weather API with type of alert
 - Fields: district, alertType, description, rainMm , createdAt
5. **Analytics Collection** –Stores the mobile apps vitals and major metrics like usercount , growth etc,
 - Fields: userId, complaintsResolved , complaintsCreated , newUsers

Figure 1. Database Collections (Conceptual Overview)



Database Choice and Connection

As mentioned above, there are two deployment options for MongoDB: local server and cloud-hosted (MongoDB Atlas).

For this project, the MongoDB Atlas cloud version was selected because of its ease of deployment, automatic backups, and free-tier availability.

MongoDB can be connected in three ways:

- Using MongoDB Shell
- Through MongoDB Compass
- By connecting the Node.js application via a connection string

For this project, the application connection method was chosen.

Connecting to MongoDB in Node.js

Below is the standard setup used to connect the Node.js server to MongoDB using **Mongoose**:

```
import mongoose from "mongoose"

import dotenv from "dotenv"

dotenv.config();

export const connectDB = async (db : string | undefined) : Promise<void> => {
  try {
    if(!db){
      process.exit(1);
    }
    await mongoose.connect(db);
    console.log("✔ Connected DB successfully")
  }
  catch(error){
    console.error("Error connecting DB",error)
    process.exit(1);
  }
}
```

```

dotenv.config();

const app = express();

app.use(cors());

app.use(express.json());

app.use("/api/auth", authRouter);
app.use("/api/schemes", schemeRouter);
app.use("/api/complaints", complaintRouter);
app.use("/api/ai", aichatRouter);
app.use("/api/location", isAuth, serviceLocatorRouter);
app.use("/api/user", isAuth, userRouter);
app.use("/api/districts", districtsRouter);
app.use("/api/alerts", AlertRouter);

const PORT : string | number = process.env.BACKEND_PORT || 5000;

connectDB(process.env.MONGO_URI).then(() => {
  app.listen(PORT, () => {
    console.log(`Server running on ${PORT}`)
  })
  runAlertFetcher();

  cron.schedule('*/*30 * * * *', () => {
    console.log('⌚ Cron: running alert fetcher...');
    runAlertFetcher();
  });
})

```

The connection string (MONGO_URI) is stored securely in the .env file provided by **MongoDB Atlas**, for example:

```

BACKEND_PORT = 5000
MONGO_URI = mongodb://localhost:27017/Keralaim

```

Schema and Model Design

Each MongoDB collection is defined using **Mongoose schemas** that specify the document structure, field types, and validation rules.

Example: User Schema

Each schema is then converted into a **model**, allowing the server to perform CRUD operations on that collection.

```

const userSchema = new mongoose.Schema({
  name: {
    type: String,
    required: true,
  },
  email: {
    type: String,
    required: true,
    unique: true,
  },
  password: {
    type: String,
    required: true,
  },
  isAdmin: {
    type: Boolean,
    required: true,
    default: false,
  },
  district: String,
  dob: Date,
  userDetails: { type: userDetailsSchema },
  bookMarkedSchemes: [mongoose.Schema.ObjectId],
  lastLoggedIn: Date,
}, { timestamps: true });

export type UsersType = mongoose.InferSchemaType<typeof userSchema> & {
  _id: Types.ObjectId;
};

```

API Route Implementation

After defining the models, the **API routes** were created as described in **Table 1**. Routes were organized into separate files by functionality — for example:

- authRoutes.js for authentication
- userRoutes.js for profile management
- complaintRoutes.js for creating complaints
- schemesRoutes.js for scheme recommendation
- serviceLocatorRoutes.js for getting nearby services
- disasterAlertRoutes.js for getting disaster alerts
- adminRoute.js for admin management

Each API route was assigned a specific HTTP method based on its function:

- **GET** – Retrieve data from MongoDB
- **POST** – Create new data entries
- **PUT** – Update existing records
- **DELETE** – Remove data

Example: Route for Complaints

```
import express from "express";
import { createComplaint, getUserComplaints } from "../controllers/complaintController";
import { upload } from "../middleware/upload";
import { isAuth } from "../middleware/authMiddleware";

const complaintRouter = express.Router();

complaintRouter.post("/", isAuth, upload.single("image"), createComplaint);
complaintRouter.get("/user", isAuth, getUserComplaints);
export default complaintRouter;
```

This route retrieves create and get complaints of particular user.

If the operation is successful, the response includes success: true and the data array.

If an error occurs, an appropriate error message is returned.

Example: POST Route to Register a New User

```
import express from "express";
import { login, register } from "../controllers/authControllers";

const authRouter = express.Router();

authRouter.post("/login", login);
authRouter.post("/register", register);

export default authRouter
```

```

export const register = async (req: Request<{}, {}, RegisterBody>, res: Response<ApiResponse<RegisterResponseBody>>) => {
  try{
    const { name, district, email, password } = req.body;
    console.log(name,district,email,password);
    if (!name || !district || !email || !password) return res.status(400).json({ message: "All fields required ", success: false });

    const existingUser = await Users.findOne({ email });

    if (existingUser) {
      return res.status(400).json({
        success: false,
        message: "User already exists",
      });
    }

    const salt = await bcrypt.genSalt();
    const hashedPassword = await bcrypt.hash(password, salt);

    const newUser = await Users.create({
      name: name,
      email: email,
      password: hashedPassword,
      district: district,
    });

    const token = jwt.sign({
      userId: newUser._id,
      isAdmin: newUser.isAdmin
    }, process.env.JWT_SECRET as string)

    return res.status(201).json({ message: "registered succesfully ", success: true, data: { userId: newUser._id, token: token } })
  }
  catch(error){
    console.error("Error : ",error)
  }
}

```

This route helps to register new users using our application. The password is not stored directly inside the DB , instead it is crypted and salted with bcrypt package.

Error Handling

All async routes are wrapped in a middleware to handle unhandled promise rejections uniformly:

```

backend > src > middlewares > JS errorhandler.js > errorhandler > errorhandler
1  exports.errorHandler = (err, req, res, next) => {
2    console.error(err);
3    const status = err.statusCode || 500;
4    res.status(status).json({message : err.message
5      || "Server Error"
6    })
7  }

```

6.3 Mobile application implementation

This section explains the implementation of the mobile app. The app is built with React Native, and targets mobile and tablets. The goal is fast development, modular components, and a smooth interactive experience

6.3.1 Tech choices & why

- **React Native** — component-based, huge ecosystem, fast iterative development.
- **Expo** — fast dev server, instant HMR for React.
- **Native CSS** — utility-first styling for consistent, fast UI development.
- **Expo Router** — file based routing build in expo
- **React Query (TanStack Query)** — data fetching, caching, background re-fetching, optimistic updates.
- **React Hook Form** — performant form handling and validation.
- **Axios** — HTTP client.
- **React Testing Library + Jest + MSW/Supertest (backend)** — tests for components and integration.

6.3.2 Project initialization

Initialize with Expo:

```
npx create-expo-app@latest mobile-app
```

```
cd mobile-app
```

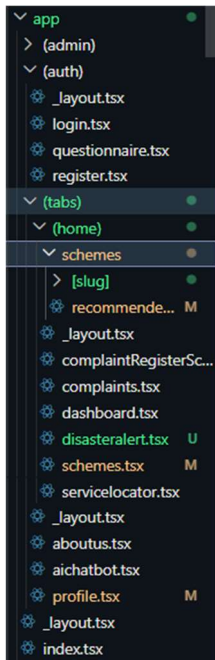
```
npx expo start
```

Run the App:

- On a physical device: Open the Expo Go app and scan the QR code. On iOS, you can use the default Camera app to scan it.
- On an Emulator/Simulator: Press a for Android or i for iOS in the terminal window.
- On the Web: Press w to open the app in your browser.

6.3.3 Routing and navigation, folder structure

Use expo-router to implement routes, It uses file bases routing



6.3.4 Global API client and auth

Create central axios instance that sets base URL and attaches JWT token from localStorage.

```
import axios from "axios";
import * as SecureStore from "expo-secure-store";

const api = axios.create({
  baseURL: "http://192.168.18.8:5000/api",
  timeout: 30000,
  headers: {"Content-Type": "application/json"}
});

api.interceptors.request.use(
  async (config) => {
    const token = await SecureStore.getItemAsync("token");

    if (token) {
      config.headers.Authorization = `Bearer ${token}`;
    }

    return config;
  },
  (error) => Promise.reject(error)
);

api.interceptors.response.use(
  (res) => res,
  (error) => {
    const backendData = error?.response?.data
    error.customMessage = backendData || "Server Error"
    error.success = backendData?.success ?? false
    return Promise.reject(error)
  }
);

export default api
```

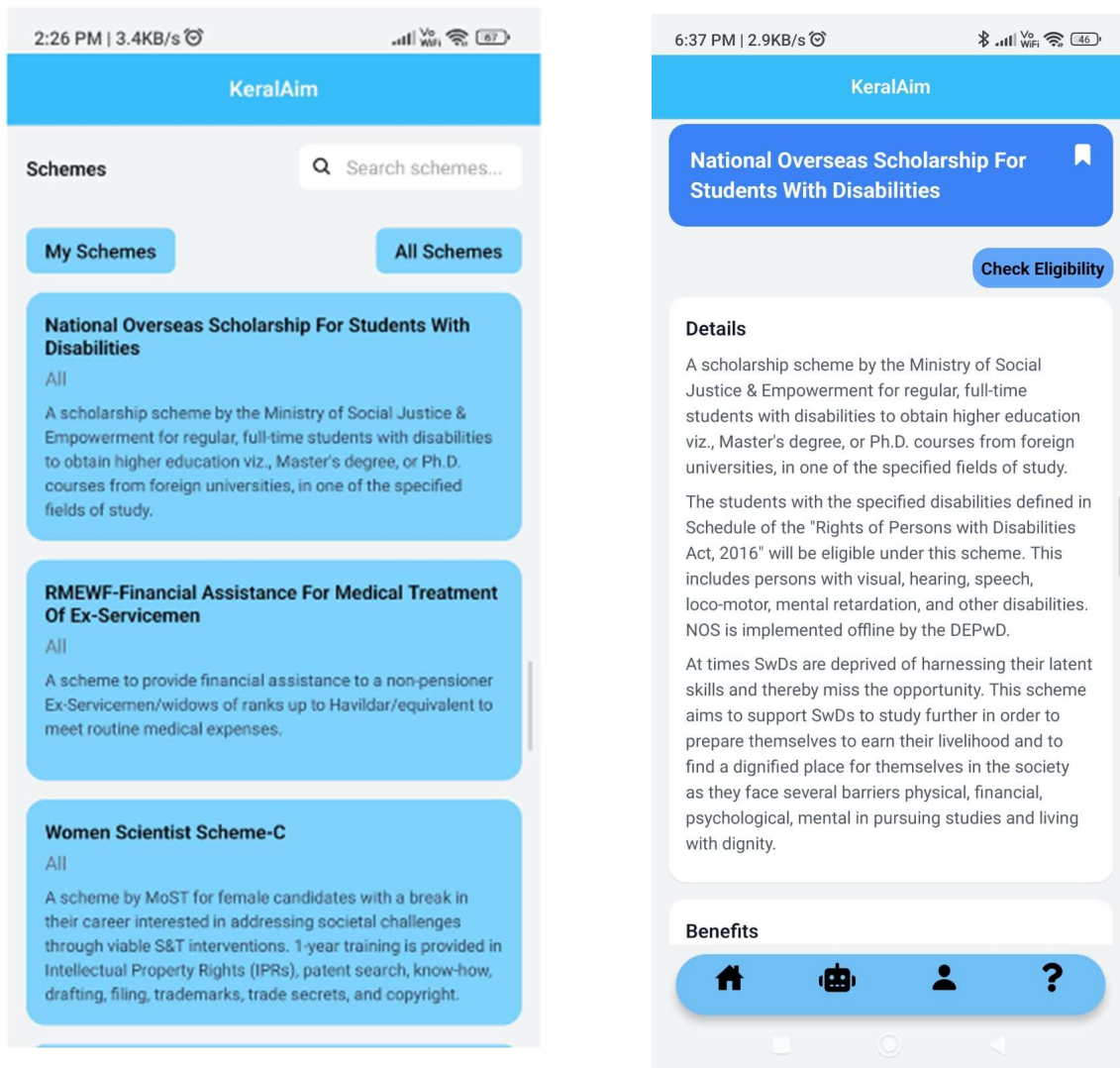
6.3.5 Key Screens & components

The main pages and components and their responsibilities.

SchemesScreen

- Fetches paginated schemes via useSchemes hook (React Query).
- Filters: search, category. Debounces input to avoid rapid requests.
- Displays grid of SchemeCard components.
- Each SchemeCard shows title, short details, level

UX: use skeleton loaders while fetching. Use optimistic update for enroll action.



```

export default function SchemesScreen() {
  const insets = useSafeAreaInsets();
  const [pageNumber, setPageNumber] = useState(1);
  const { isLoading, data, isError, isSuccess } = useSchemes(pageNumber);
  if (isError) {
    return <Text>Error loading schemes</Text>;
  }
  return (
    <View
      className="flex-1 bg-gray-100"
      style={{ paddingTop: insets.top, paddingBottom: insets.bottom + 60 }}
    >
      { /* Header */ }
      <AppBar />
      <ScrollView className="px-4 mt-4">
        { /* Title */ }
        <Text className="text-base font-semibold mb-3">
          Schemes
        </Text>

        { /* Toggle Buttons */ }
        <View className="flex-row justify-between mb-4">
          <TouchableOpacity className="bg-sky-300 px-4 py-2 rounded-lg">
            <Text className="font-medium">My Schemes</Text>
          </TouchableOpacity>

          <TouchableOpacity className="bg-sky-300 px-4 py-2 rounded-lg">
            <Text className="font-medium">All Schemes</Text>
          </TouchableOpacity>
        </View>

        {isLoading ? (
          <>
            {[1, 2, 3, 4].map((i) => (
              <SchemeCardSkeleton key={i} />
            ))}
          </>
        ) : isError ? (
          <Text>Error loading schemes</Text>
        ) : (
          data?.data?.map((scheme: any) => (
            <SchemeCard key={scheme.slug} scheme={scheme} />
          ))
        )}

        {
          data?.totalPages > 1 && <Pagination
            totalPages={data.totalPages}
            currentPage={pageNumber}
            onPageChange={setPageNumber}
          />
        }
      </ScrollView>
    </View>
  );
}

```

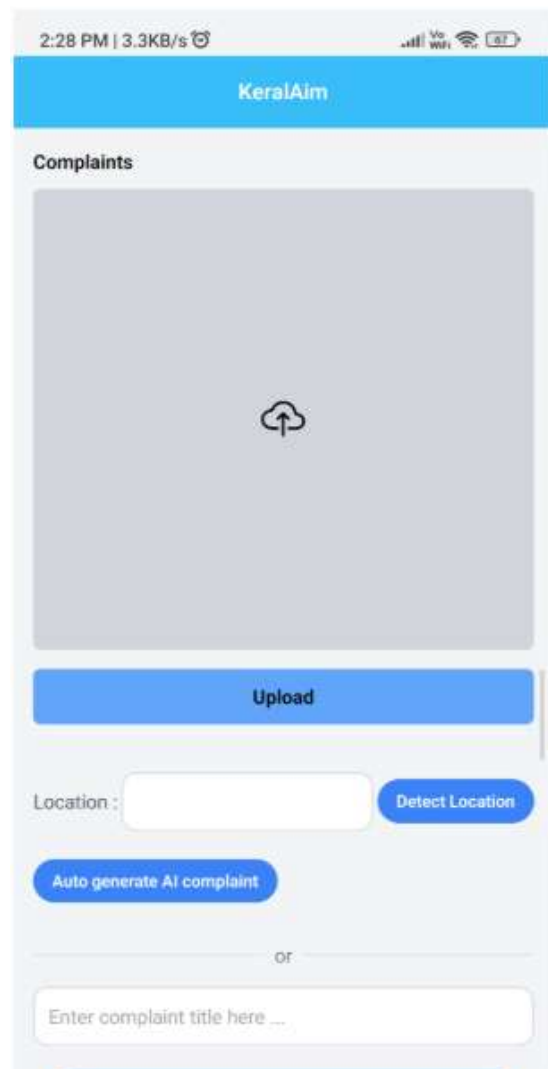
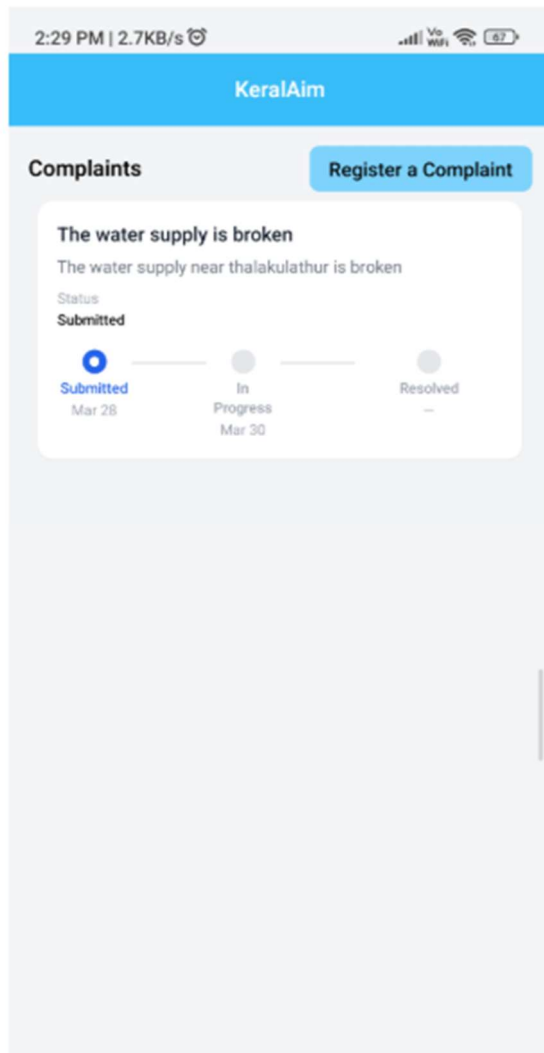
ComplaintsScreen

- Fetches /complaints/user
- Shows list of user registered complaint and their status
- A button to create new complaint that takes to createComplaint Screen

```
import { ComplaintCard } from "@components/complaintsCard";
import { useComplaints } from "@hooks/useComplaints";
import { useRouter } from "expo-router";
import { FlatList, Pressable, Text, View } from "react-native";
import { useSafeAreaInsets } from "react-native-safe-area-context";

type Complaint = { _id: string; title: string; description: string };

export default function ComplaintScreen(){
  const insets = useSafeAreaInsets();
  const router = useRouter();
  const {isLoading, isError, data, error} = useComplaints();
  console.log("ComplaintScreen data:", data, "error:", error);
  return(
    <>
      <View
        className="flex-1 bg-gray-100"
        style={{ paddingTop: insets.top, paddingBottom: insets.bottom + 60 }}
      >
        <AppBar>
          <View className="p-4">
            <View className="flex-row justify-between items-center">
              <Text className="text-lg font-semibold">Complaints</Text>
              <Pressable className="bg-sky-300 px-4 py-2 rounded-lg" onPress={() => router.push("/complaintRegisterScreen")}>
                <Text className="font-medium">Register a Complaint</Text>
              </Pressable>
            </View>
          </View>
          {
            isLoading ? <Text className="text-center mt-4">Loading complaints...</Text> :
            isError ? <Text className="text-center mt-4">Error loading complaints: {error.message}</Text> :
            data.length === 0 ? <Text className="text-center mt-4">No complaints found.</Text> :
            <FlatList
              contentContainerStyle={{ paddingBottom: insets.bottom + 20 }}
              data={data}
              renderItem={({ item }: { item: Complaint }) => (
                <ComplaintCard item={item} />
              )}
              keyExtractor={(item: Complaint) => item._id.toString()}
            />
          }
        </View>
      </View>
    </>
  )
}
```



ServiceLocatorScreen

- Displays all nearby services categories to types

```
export default function ServiceLocatorScreen() {
  const insets = useSafeAreaInsets();

  const [currentLocation, setCurrentLocation] = useState({ lat: number; lng: number } | null>(null);
  const [selectedCategory, setSelectedCategory] = useState(categories);
  const { isLoading, isError, data, error } = useLocation(currentLocation?.lat || 0, currentLocation?.lng || 0, selectedCategory.find(c => c.isSelected)?.tag || "hospital");
  console.log("ServiceLocatorScreen data:", data, "error:", error);
  const openMap = (lat: number, lng: number) => {
    const url = "https://www.google.com/maps/dir/?api=1&destination=${lat},${lng}";
    Linking.openURL(url);
  };

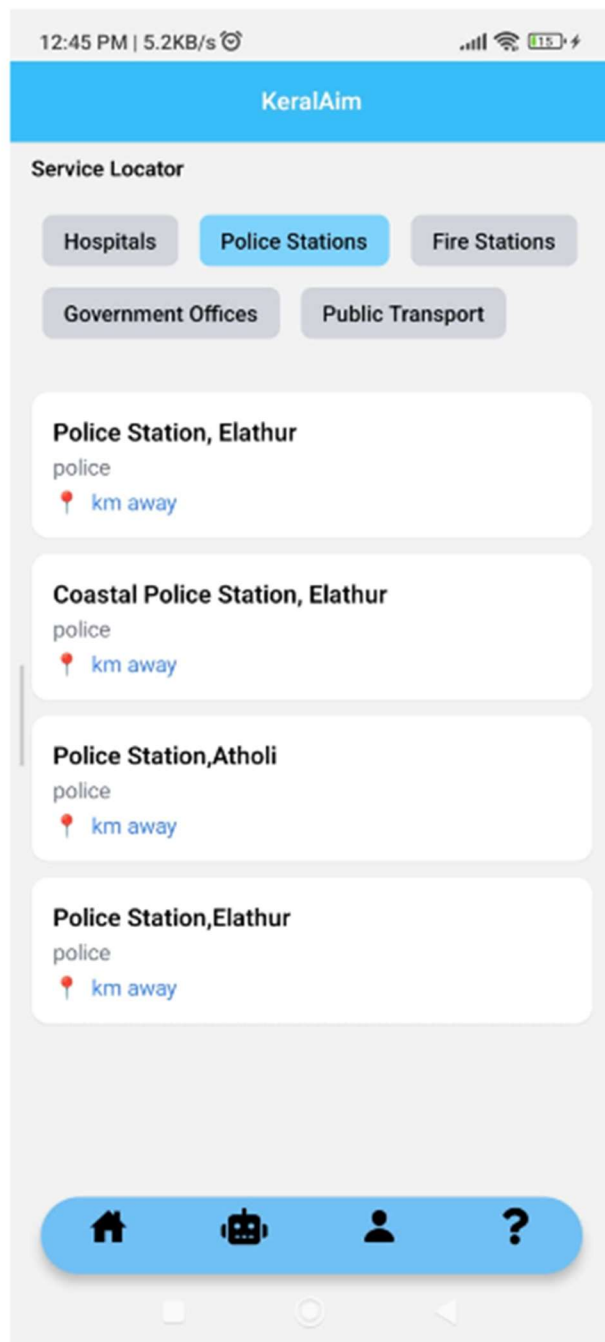
  const handleCategoryChange = (tag: string) => {
    setSelectedCategory(prev =>
      prev.map(category =>
        category.tag === tag
          ? { ...category, isSelected: true }
          : { ...category, isSelected: false }
      )
    );
  };

  useEffect(() => {
    const fetchLocationData = async () => {
      const { status } = await Location.requestForegroundPermissionsAsync();
      if (status !== "granted") return;

      const loc = await Location.getCurrentPositionAsync();
      setCurrentLocation({
        lat: loc.coords.latitude,
        lng: loc.coords.longitude,
      });
    };

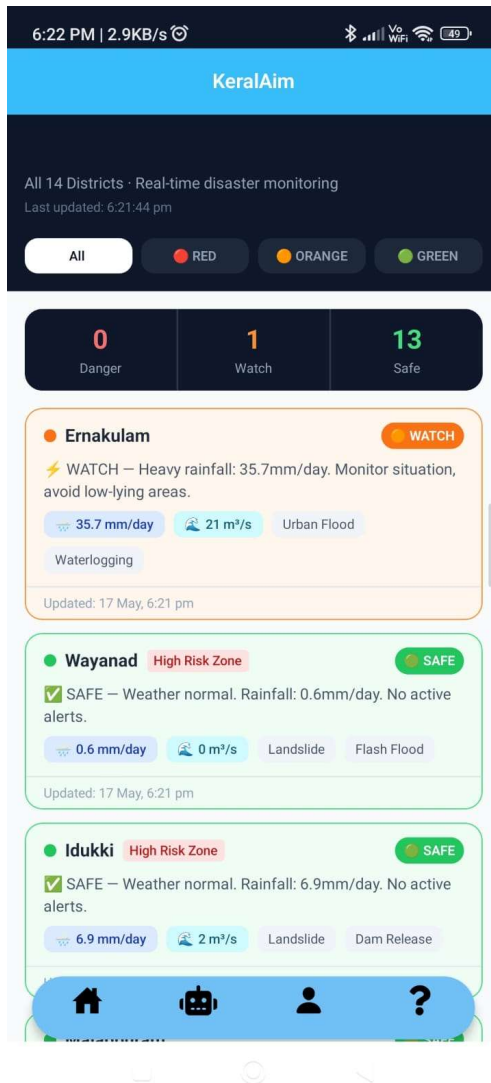
    fetchLocationData();
  }, []);
}
```

```
if (isError) {
  return <Text>Error fetching location data</Text>
}
return (
  <View className="flex-1" style={{ padding: insets.top, padding: insets.bottom + 60 }}>
    <AppBar />
    <Text className="font-sans font-size-16" style={{ padding: 10 0 0 0 }}>Service Locator</Text>
    <View className="flex-row flex-wrap p-4">
      {
        selectedCategory.map((category) => (
          <Pressable key={category.tag} className="px-4 py-2 m-2 rounded-lg ${category.isSelected ? "bg-sky-300" : "bg-gray-300"}>
            <Text className="font-medium" style={{ padding: 0 10 0 0 }}>{category.name}</Text>
          </Pressable>
        ))
      }
    </View>
    <ScrollView className="flex-1 p-4">
      <View>
        <Text>{isLoading ? "Loading..." : "Error"}</Text>
        <View>
          <Text>{data?.length ? "Found " + data.length + " results" : "No results found"}</Text>
          <View>
            {
              data?.map(place => (
                <TouchableOpacity>
                  <Text>{place.id}</Text>
                  <Text>{place.name}</Text>
                  <Text>{place.type}</Text>
                  <Text>{place.distance} km away</Text>
                </TouchableOpacity>
              ))
            }
          </View>
        </View>
      </ScrollView>
    </View>
  )
)
```



Disaster Alert Screen

Displays all the alerts of 14 districts and categorize them , auto fetches every 30s



```

export default function HomeScreen() {
  return () => {
    <View className="flex-1 bg-slate-50">
      <StatusBar barStyle="light-content" backgroundColor="#0f172a" />
      <AppBar>
        { /* --- Header --- */ }
        <View className="bg-slate-900 pt-12 pb-5 px-4">
          <Text className="text-slate-400 text-sm mt-1">
            All 14 Districts · Real-time disaster monitoring
          </Text>
          <Text>
            {lastFetch && {
              <Text className="text-slate-500 text-xs mt-1">
                Last updated: {lastFetch.toLocaleTimeString('en-IN', { hour12: true })}
              </Text>
            }}
          </Text>
        </View>

        { /* --- Filter tabs --- */ }
        <View className="flex-row bg-slate-900 pb-4 px-4 gap-2">
          {['ALL', 'RED', 'ORANGE', 'GREEN'].map((lv1) => {
            <TouchableOpacity>
              {key=lv1,
               onPress={() => setFilter(lv1)},
               className: {
                 flex-1 items-center py-2 rounded-xl
                 ${filter === lv1 ? 'bg-white' : 'bg-slate-800'}
               }
            }
            <Text>
              {className: 'text-xs font-semibold',
               ${filter === lv1 ? 'text-slate-900' : 'text-slate-400'}
            }
            </Text>
            {lv1 === 'ALL' ? 'All' : lv1 === 'RED' ? '🔴' : lv1 === 'ORANGE' ? '🟠' : '🟢'}{ ' ' }
            {lv1 !== 'ALL' && lv1}
          </Text>
          </TouchableOpacity>
        )}
        </View>

        <ScrollView>
          {className="flex-1",
           showsVerticalScrollIndicator=(false),
           refreshControl: {
             <RefreshControl>
               {refreshing=(refreshing),
                onRefresh={() => fetchData(true)},
                colors=[ '#EF4444' ],
                tintColor="#EF4444"
              }
            }
          }
        </ScrollView>
      </View>
    }
  }
}

```

```

    <View className="pt-4" />

    { /* Summary counts */ }
    {districts.length > 0 && <SummaryBar districts={districts} />}

    { /* Error banner */ }
    {error && {
      <View className="mx-4 mb-4 rounded-xl bg-red-100 border border-red-300 px-4 py-3">
        <Text className="text-red-700 text-sm">{error}</Text>
        <TouchableOpacity onPress={() => fetchData()} className="mt-2">
          <Text className="text-red-600 text-sm font-semibold">Tap to retry</Text>
        </TouchableOpacity>
      </View>
    })}

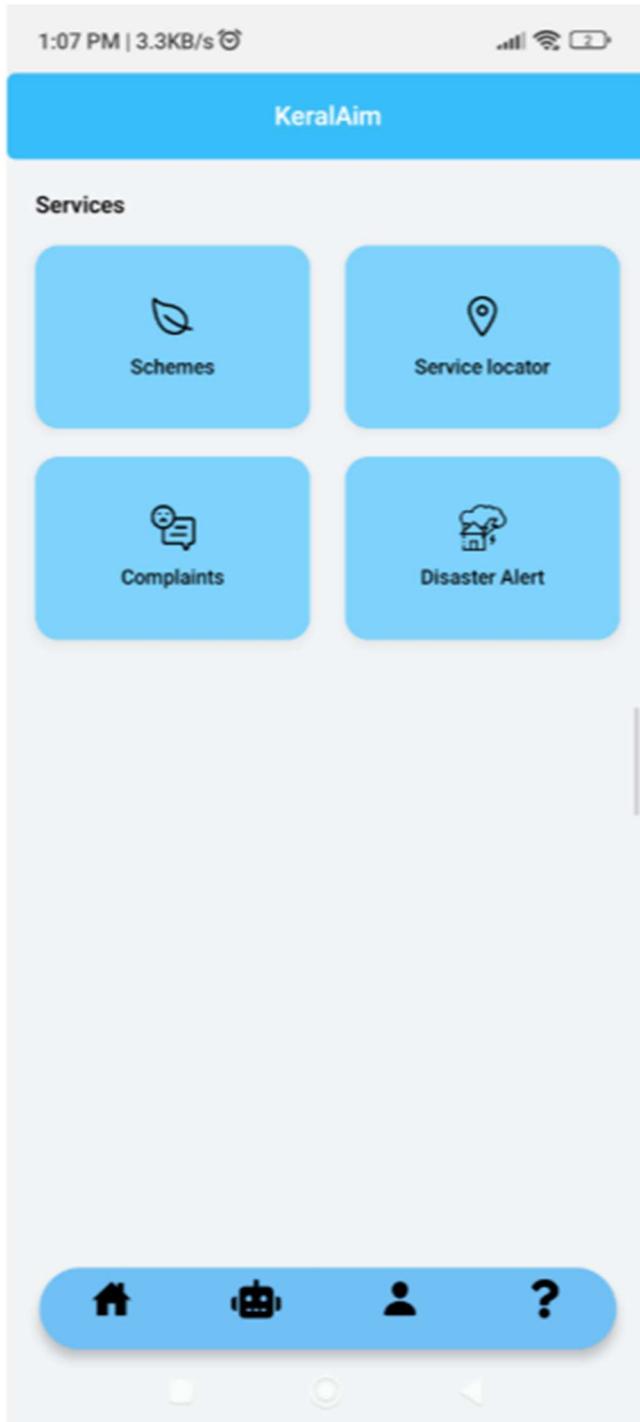
    { /* District cards */ }
    {filtered.length === 0 ? {
      <View className="items-center py-16">
        <Text className="text-slate-400 text-base">No districts match this filter</Text>
      </View>
    } : {
      filtered.map((item, index) => {
        <DistrictRow key={item.district} item={item} isLast={index === filtered.length - 1} />
      })
    })}

    { /* Footer */ }
    <View className="items-center py-6">
      <Text className="text-slate-400 text-xs">
        Data: Open-Meteo · GDACS (UN) · GloFAS
      </Text>
      <Text className="text-slate-400 text-xs mt-1">
        Auto-refreshes every 30 min
      </Text>
    </View>
  </ScrollView>
</View>
);
}

```

User Dashboard

- Shows the Schemes,Complaint,Disaster,ServiceLocator Tiles
- The user can choose any as per their wish



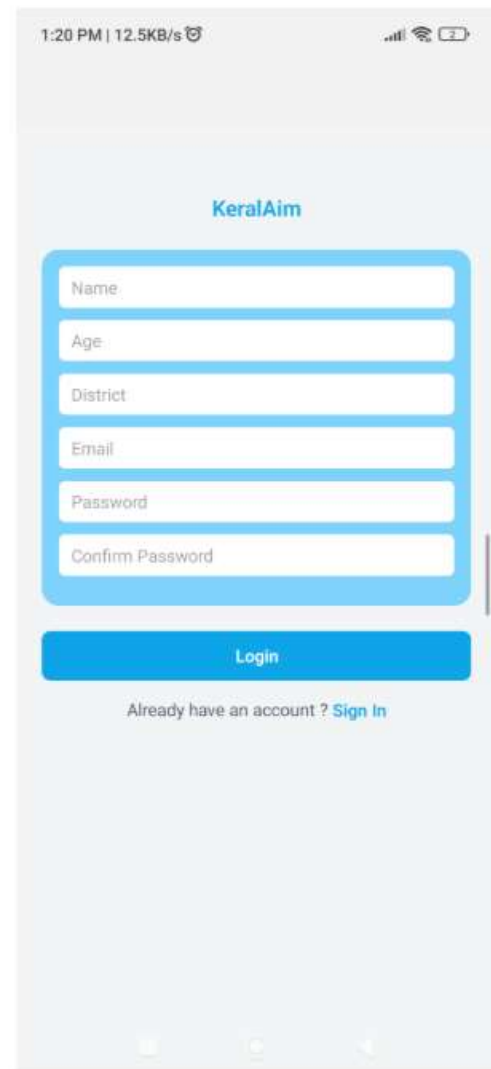
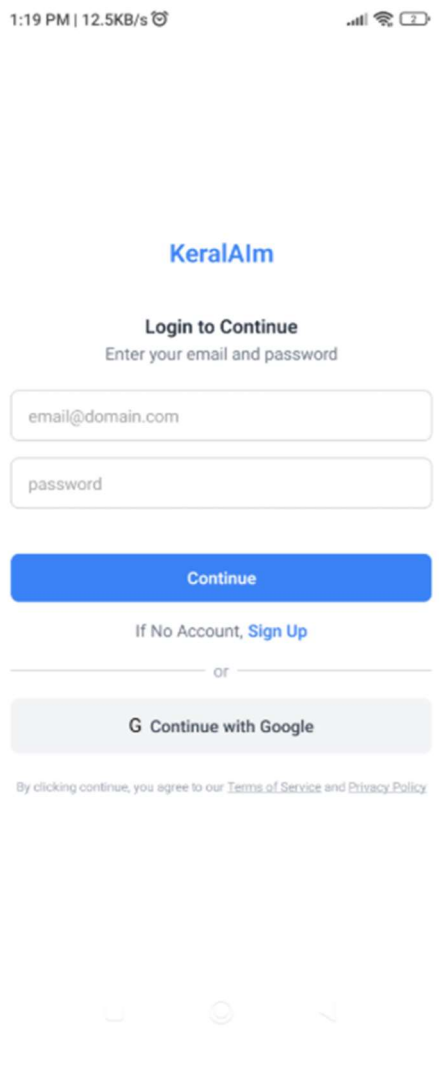
6.3.6 Forms & inputs

- **React Hook Form** for forms: fast, minimal rerenders.
- Use validation rules and show inline errors.
- Uses yup and yup resolver with react hook form for validation process.

```
import React, {useState} from 'react';
import { useForm } from 'react-hook-form';
import { yupResolver } from '@hookform/resolvers/yup';
import * as yup from 'yup';
import { registeruser } from '../../services/api';
import { useNavigate } from 'react-router-dom';

const schema = yup.object().shape({
  name: yup.string().min(2, "At least two character").required('Name required'),
  email: yup.string().email('Invalid email').required('Email Required').matches(/^(?!.*\.\.)(?!.*\.$)[a-zA-Z0-9-]+\@gmail\.com$/,
  password: yup.string().min(8, "Atleast 8 charcter").required("Password required").matches(/^(?=.*[a-z].*)(?=.*[A-Z].*)(?=.*[0-9].*)(?=.*[\W_]).{8,}$/), 'Password must contain at least one uppercase letters'),
  confirm: yup.string().oneOf([yup.ref('password')], "Password must match").required('Confirm Required')
});

export default function RegisterForm() {
  const {register,handleSubmit,formState:{errors,isSubmitting}} = useForm({resolver: yupResolver(schema)})
  const [serverError,setServerError] = useState('');
  const nav = useNavigate();
  const onSubmit = async (data) => {
    setServerError('');
    const payload = {name: data.name, email: data.email, password: data.password};
    try {
      const res = await registeruser(payload)
      localStorage.setItem('token',res.data.token);
      nav('/register/question');
    } catch(error) {
      setServerError(error?.response?.data?.message || "Registration Failed");
    }
  };
  return(
    <>
    <form onSubmit={handleSubmit(onSubmit)} className='space-y-4'>
      {serverError && <div className='text-red-600 text-sm'>{serverError}</div>}
      <div>
        <label className='block text-sm mb-1'>Name</label>
        <input {...register('name')} className='min-w-[350px] px-3 py-2 border rounded text-black focus:outline-none focus:ring' />
        {errors.name && <p className='text-xs text-red-600 mt-1'>{errors.name.message}</p>}
      </div>
      <div>
        <label className='block text-sm mb-1'>Email</label>
        <input {...register('email')} className='min-w-[350px] px-3 py-2 border rounded text-black focus:outline-none focus:ring' />
        {errors.email && <p className='text-xs text-red-600 mt-1'>{errors.email.message}</p>}
      </div>
    </form>
  )
}
```



6.3.7 State management & caching

- Rely on **React Query** for server state (courses, recommendations, roadmap).
- Use `invalidateQueries` on mutation success (e.g., enroll, create course).
- Local UI state for forms, modals and selected node in roadmap.

Benefit: less boilerplate for caching and refetching; optimistic updates are easy.

```

export const useComplaints = () => {
  const queryClient = useQueryClient();
  useFocusEffect(
    useCallback(() => {
      queryClient.invalidateQueries({ queryKey: ['complaints'] });
    }, [])
  );
  return useQuery({
    queryKey: ["complaints"],
    queryFn: () => getAllUserComplaints(),
    staleTime: 1000 * 60 * 5,
    retry: 1
  })
}

```

6.3.8 Accessibility & UX

- Buttons have accessible labels and keyboard focus states.
- Use semantic HTML (buttons, headings, lists).
- Provide loading text and spinners.
- Provide clear inline error messages on form submission.

6.3.9 Example code snippets (small helpers)

Debounce hook (used in SchemeList for search): This will help to wait for some time after user key stroke is completed and then search for the course to optimize repeated calls during each key stroke.

```

import { useEffect, useState } from "react";

export default function useDebounce(value, delay = 1000) {
  const [debounce, setDebounce] = useState(value);
  useEffect(() => {
    const t = setTimeout(() => setDebounce(value), delay);
    return () => clearTimeout(t);
  }, [value, delay]);
  return debounce;
}

```

6.3.10 Example reused Components (Pagination and navbars)

Pagination

It render a neat and cool UI which help users to navigate through hundreds of schemes and help the backend to fetch only required scheme to show to users. Helps in efficiency

```
const Pagination: React.FC<PaginationProps> = ({
  totalPages,
  currentPage,
  onPageChange,
}) => {
  const pages = Array.from({ length: totalPages }, (_, i) => i + 1);

  return (
    <View className="flex-row items-center gap-2 p-3">
      {/* Prev Button */}
      <TouchableOpacity>
        disabled={currentPage === 1}
        onPress={() => onPageChange(currentPage - 1)}
        className={`px-3 py-2 rounded-md bg-gray-200 ${
          currentPage === 1 ? "opacity-40" : ""
        }`}
      </TouchableOpacity>
      <Text className="font-semibold text-gray-800">Prev</Text>

      {/* Page Numbers */}
      <ScrollView>
        horizontal
        showsHorizontalScrollIndicator={false}
        className="flex-row"
      </ScrollView>
      {pages.map((page) => (
        <TouchableOpacity>
          key={page}
          onPress={() => onPageChange(page)}
          className={`mx-1 px-4 py-2 rounded-md ${
            currentPage === page ? "bg-blue-500" : "bg-gray-200"
          }`}
        </TouchableOpacity>
        <Text>
          {currentPage === page ? "text-white font-bold" : "text-gray-800"}
        </Text>
        {page}
      ))}

      {/* Next Button */}
      <TouchableOpacity>
        disabled={currentPage === totalPages}
        onPress={() => onPageChange(currentPage + 1)}
        className={`px-3 py-2 rounded-md bg-gray-200 ${
          currentPage === totalPages ? "opacity-40" : ""
        }`}
      </TouchableOpacity>
      <Text className="font-semibold text-gray-800">Next</Text>
    </View>
  );
};

export default Pagination;
```

Bottom Nav Bar : Users can easily navigate to main part of the app

```

export default function TabLayout() {
  const insets = useSafeAreaInsets();
  return (
    <Tabs screenOptions={{
      headerShown: false,
      tabBarShowLabel: false,
      tabBarStyle: {
        backgroundColor: "#71C0F5",
        height: 50,
        bottom: insets.bottom + 2,
        borderRadius: 30,
        marginHorizontal: 20,
        position: "absolute",
        borderTopWidth: 0,
        elevation: 5, // Android
        shadowColor: "#000",
        shadowOpacity: 0.1,
        shadowRadius: 10,
      },
      tabBarActiveTintColor: '#ffd33d',
      tabBarInactiveTintColor: '#cccccc',
    }}>
      <Tabs.Screen name="(home)" options={{
        title: "Home",
        tabBarIcon: () => (
          <Ionicons name="home-sharp" size={24} color="black" />
        )
      }} />
      <Tabs.Screen name="aichatbot" options={{
        title: "AI Chat Bot",
        tabBarIcon: () => (
          <FontAwesome6 name="robot" size={24} color="black" />
        )
      }} />
      <Tabs.Screen name="profile" options={{
        title: "Profile",
        tabBarIcon: () => (
          <Ionicons name="person" size={24} color="black" />
        )
      }} />
      <Tabs.Screen name="aboutus" options={{
        title: "About Us",
        tabBarIcon: () => (
          <FontAwesome5 name="question" size={24} color="black" />
        )
      }} />
    </Tabs>
  )
}

```

6.3.11 Main functionalities summary

- **Authentication:** Register/login/logout using JWT; protected routes for admin.
- **Course browsing:** Search, filter, pagination, view course details.
- **Enrollments:** Enroll/unenroll, progress tracking.
- **Recommendations:** Personalized fill-gaps and reinforce views, exclude already enrolled courses.
- **Roadmap generation:** AI-based generation, saved to DB, editable graph.
- **Admin:** create/update/delete courses; manage users and roles.
- **Analytics:** basic learning progress, % completion aggregation (front/back integration).
- **Testing & CI:** unit tests, integration, deployment pipeline.

7. Testing

System Testing is a level of testing that validates the complete and fully integrated software product. The purpose of a system test is to evaluate the end-to-end system specifications. Usually, the software is only one element of a larger computer-based system. Ultimately, the software is interfaced with other software/hardware systems. System Testing is defined as a series of different tests whose sole purpose is to exercise the full computer-based system

7.1 Testing methodology

MANUAL TESTING

Manual Testing is a type of software testing in which test cases are executed manually by a tester without using any automated tools. The purpose of Manual Testing is to identify the bugs, issues, and defects in the software application. Manual software testing is the most primitive technique of all testing types and it helps to find critical bugs in the software application.

Any new application must be manually tested before its testing can be automated. Manual Software Testing requires more effort but is necessary to check automation feasibility. Manual Testing concepts does not require knowledge of any testing tool.

AUTOMATION TESTING

Automation Testing is a software testing technique that performs using special automated testing software tools to execute a test case suite. On the contrary, Manual Testing is performed by a human sitting in front of a computer carefully executing the test steps.

The automation testing software can also enter test data into the System Under Test, compare expected and actual results and generate detailed test reports. Software Test Automation demands considerable investments of money and resources.

7.1.2 DIFFERENT TESTING

UNIT TESTING

Unit Testing is a type of software testing where individual units or components of a software are tested. The purpose is to validate that each unit of the software code performs as expected. Unit Testing is done during the development (coding phase) of an application by the developers. Unit Tests isolate a section of code and verify its correctness. A unit may be an individual function, method, procedure, module, or object.

INTEGRATION TESTING

Integration Testing is defined as a type of testing where software modules are integrated logically and tested as a group. A typical software project consists of multiple software modules, coded by different programmers. The purpose of this level of testing is to expose defects in the interaction between these software modules when they are integrated

VALIDATION TESTING

Validation testing is the process of ensuring that the tested and developed software satisfies the client /user's needs. The business requirement logic or scenarios have to be tested in detail. All the critical functionalities of an application must be tested here.

As a tester, it is always important to know how to verify the business logic or scenarios that are given to you. One such method that helps in detailed evaluation of the functionalities is the Validation Process.

Front-End Validation

Front End Testing validates that said GUI elements are functioning as expected. For example, it checks if input fields accept the right characters, if forms are being submitted only after the required fields are filled, if navigation is easy enough, if the page is loading fast enough, etc.

The goal of Front-End Testing is to test functionalities and verify that a website or app's presentation layer is bug or error-free. This has to be done after every single update to ensure recent changes have not degraded any aspect of the UI

Back-End Validation

Back-end Validation is a testing method that checks the database or server-side of the web application. The main purpose of backend testing is to check the application layer and the database layer. It will find an error or bug in the database or server-side. It is also known as Database testing. Backend testing is a must because anything wrong or error happens at the server-side, it will not further proceed with that task or the output will get differed or sometimes it will also cause problems such as data loss, deadlock, etc.,

OUTPUT TESTING

Output Testing is performed after validation. It shows whether our system produce the output which is appropriate to our input. If this test is neglected then our system's efficiency might be decreased thus causing the working of the system. Because of this we must take care while doing output testing.

7.2 TEST CASE AND OUTPUTS

7.2.1 UNIT TEST

1) Function Complaint Service

This function will route the complaints based on the text provided to its correct department like water authority , municipalities , PWD etc .

So it is crucial to test the functionality of this function is working properly as planned. Bellow is the test output and sample test code.

```
PASS src/test/complaintServices.test.ts
Complaint Classification
  ✓ should classify road issues (3 ms)
  ✓ should classify water issues (1 ms)
  ✓ should return general category (1 ms)

Test Suites: 1 passed, 1 total
Tests:       3 passed, 3 total
Snapshots:  0 total
Time:        1.037 s
Ran all test suites matching ./src/test/complaintServices.test.ts.
```

```
import { classifyComplaint } from "../services/complaintServices";

describe("Complaint Classification", () => {
  it("should classify road issues", () => {
    const result = classifyComplaint("road is broken");
    expect(result).toBe("Infrastructure");
  });

  it("should classify water issues", () => {
    const result = classifyComplaint("water leak");
    expect(result).toBe("Water");
  });

  it("should return general category", () => {
    const result = classifyComplaint("random issue");
    expect(result).toBe("General");
  });
});
```

7.2.2 Integration Test

1) Complaint Test

In this test we are integrating the frontend with backend for complaint module. Here when a user using our app can create a complaint in complaint database model but when user has to authorization to create complaint.

```
import request from "supertest";
import app from "../../src/app";

describe("Complaint API", () => {
  let token: string;

  beforeAll(async () => {
    // login user
    const res = await request(app).post("/api/auth/login").send({
      email: "test@gmail.com",
      password: "123456",
    });

    token = res.body.data.token;
  });

  it("should create a complaint", async () => {
    const res = await request(app)
      .post("/api/complaints")
      .set("Authorization", `Bearer ${token}`)
      .send({
        title: "Road issue",
        description: "Bad road",
        location: "Kerala",
      });

    expect(res.statusCode).toBe(201);
  });

  it("should fetch complaints", async () => {
    const res = await request(app)
      .get("/api/complaints/user")
      .set("Authorization", `Bearer ${token}`);

    expect(res.statusCode).toBe(200);
  });
});
```

```
PASS src/test/complaint.test.ts
Complaint API
  ✓ should create a complaint (34 ms)
  ✓ should fetch complaints (24 ms)

Test Suites: 1 passed, 1 total
Tests:       2 passed, 2 total
Snapshots:   0 total
Time:        2.08 s, estimated 3 s
Ran all test suites matching ./src/test/complaint.test.ts.
```

2) Auth Test

Next important integration testing is authentication , only users with valid username and password can use our services. So it is a must to test the authentication logic of our application . Below are the test case output and sample code.

```
PASS src/test/auth.test.ts
Auth API
  ✓ should register user (137 ms)
  ✓ should login user (64 ms)

Test Suites: 1 passed, 1 total
Tests:       2 passed, 2 total
Snapshots:   0 total
Time:        2.04 s, estimated 3 s
```

8.CONCLUSION

The objective of this mini-project was to design and implement an intelligent citizen service platform that integrates government scheme recommendations, smart complaint management, AI chatbot assistance, and location-based service discovery. Developed as a full-stack mobile application using React Native (Expo), Node.js, Express, and MongoDB, the system demonstrates the effective application of modern technologies and AI-driven personalization to simplify access to government services for citizens.

The theoretical foundation of the project focused on key software engineering principles such as system architecture design, RESTful API development, and scalable backend structuring. Core concepts including JWT-based authentication, secure data handling, database schema design, and API integration were studied and applied to ensure a robust and maintainable system. Emphasis was also given to designing a modular architecture that supports future scalability and feature expansion.

During implementation, the frontend was developed using React Native with Expo, enabling cross-platform mobile compatibility and a responsive user interface. Tailwind (NativeWind) was used for consistent styling. Features such as user authentication, scheme browsing, personalized recommendations, complaint registration with image upload, and service locator were successfully integrated. The service locator module utilized geolocation to fetch nearby government services, enhancing real-world usability.

The backend was built using Node.js and Express, providing RESTful APIs for authentication, scheme management, complaint handling, and AI chatbot functionality. JWT-based authentication ensured secure access control. Multer and Cloudinary were used for handling and storing uploaded complaint images. AI integration via OpenRouter enabled chatbot responses and intelligent parsing of scheme eligibility criteria into structured data, improving recommendation accuracy.

MongoDB was used as the primary database, allowing flexible storage of both structured and dynamic eligibility data. Advanced features such as AI-based eligibility extraction, dynamic rule evaluation, and batch processing were implemented to support scalable and efficient recommendation systems. Testing was conducted to ensure API reliability, error handling, and performance under multiple requests.

In conclusion, the Keralaim project successfully achieved its goal of creating a unified, intelligent platform that bridges the gap between citizens and government services. By combining AI capabilities with strong software engineering practices, the system provides a scalable and user-centric solution. The project lays a strong foundation for future enhancements such as direct collaboration with government, direct routing to departments , advanced AI recommendation models, cloud deployment, Native language support(Malayalam) for both text and voice and integration with real-time government data sources

9.APPENDICES

This Section include the Installation Instruction, Glossary and abbreviations for the understanding on non-technical reader . Prompt used to parse the eligibility criteria for smart recommendation by using groq api Image classification using google gemini through openrouter

9.1 Installation Instruction (Appendix A)

- **Prerequisites:**

Component	Version	Description
Node.js	$\geq 18.x$	Required for backend server
npm	$\geq 9.x$	Package manager
MongoDB	$\geq 8.x$	Database
TypeScript	$\geq 5.x$	Type safety
Git	Latest	Version control
groq	$\geq 1.1.x$	The AI API
React Native	$\geq 0.81.x$	Frontend library
Expo	$\geq 54.x$	Frontend Framework

- **Environment variable**

Variable	Description
MONGO_URI	Connection string for MongoDB
JWT	Secret key for authentication tokens
NODE_PORT	Backend server port (inside frontend folder)
GROQ_API	AI Api for parsing eligibility
OPENROUTER_API	Used in AI chatbot

Variable	Description
CLOUD_NAME	The name of cloudinary account
CLOUD_API_SECRET	Unique secret the verify the user server and cloud server
CLOUD_API_KEY	Public identifier for your Cloudinary account
FIREBASE_PRIVATE_KEY	Used for push notification

- **Setup steps:**

- Clone repo - git clone <https://github.com/krishnabackup/Keralaim.git>
- cd backend

npm install

npm run dev

- cd ../frontend

npm install

Check the ip address of the device running backend and paste that inside /services/api.ts

```
const api = axios.create({
  baseURL : "http://192.168.18.8:5000/api",
  timeout : 30000,
  headers : {"Content-Type" : "application/json"}
})
```

npm expo start

scan the qr code or open virtual device or browser

- **Run with Docker:** docker build / docker-compose up -d

9.2 Abbreviation (Appendix B.1)

Abbreviation	Full Form
API	Application Programming Interface
DB	Database
JWT	JSON Web Token
UI	User Interface
UX	User Experience
CRUD	Create, Read, Update, Delete
REST	Representational State Transfer
IDE	Integrated Development Environment
CI/CD	Continuous Integration / Continuous Deployment
JSON	JavaScript Object Notation

9.3 Glossary (Appendix B.2)

Term	Definition
API	Application Programming Interface – enables frontend-backend communication.
JWT	JSON Web Token – used for user authentication and authorization.

9.4 AI system and user prompts (Appendix C)

This section provides the prompts used to call the GROQ API , Openrouter API to parse the eligibility criteria and chatbot response respectively and

a) Eligibility Parser Function

```
export const aiEligibilityParser = async (text:string) => {
  try {
    const prompt = `
    You are an assistant for parsing eligibility criteria for government schemes in Kerala and India.
    Convert eligibility text into structured JSON.

    Return format:
    {
      "minAge": number | null,
      "maxAge": number | null,
      "incomeLimit": number | null,
      "occupation": string[],
      "caste": string[],
      "gender": string[],
      "disability": boolean,
      "disabilityPercentage": number | null,
      "isBPL": boolean,
      "confidence": number (0 to 1)
    }

    Rules:
    - Confidence should reflect how certain extraction is
    - Return ONLY JSON, no explanations or additional text.
    -Should be able to handle complex eligibility criteria and extract relevant information accurately.
    -Should also include criteria not mentioned in the return format if they are relevant to eligibility.
    -If criteria have disability and disability percentage both mentioned, include both in the response.
    -Should handle income limits mentioned in various formats (e.g., annual, monthly) and convert them to a standard format (e.g., annual income limit).
    Text : ${text}
    `

    const chatCompletion = await groq.chat.completions.create({
      messages: [{ role: "system", content: prompt }],
      model: "llama-3.1-8b-instant",
    });
    const result = chatCompletion.choices[0]?.message?.content;
    if(!result) return {error : true}
    const parsed = JSON.parse(result);
    return parsed;
  }
  catch(error){
    return { error : true}
  }
}
```

b) Chat Bot Function

```
import { openai } from "../config/openai";
export const generateReply = async (message: string) => {
  try {
    const response = await openai.chat.completions.create({
      model: "mistralai/mistral-7b-instruct-v0.1",
      messages: [
        {
          role: "system",
          content: `
You are an AI assistant for Kerala government services app (KeralaIM).

Your job:
- Suggest government schemes
- Explain application steps
- Give clear, structured answers
- Keep answers short and useful
- Prefer bullet points or steps

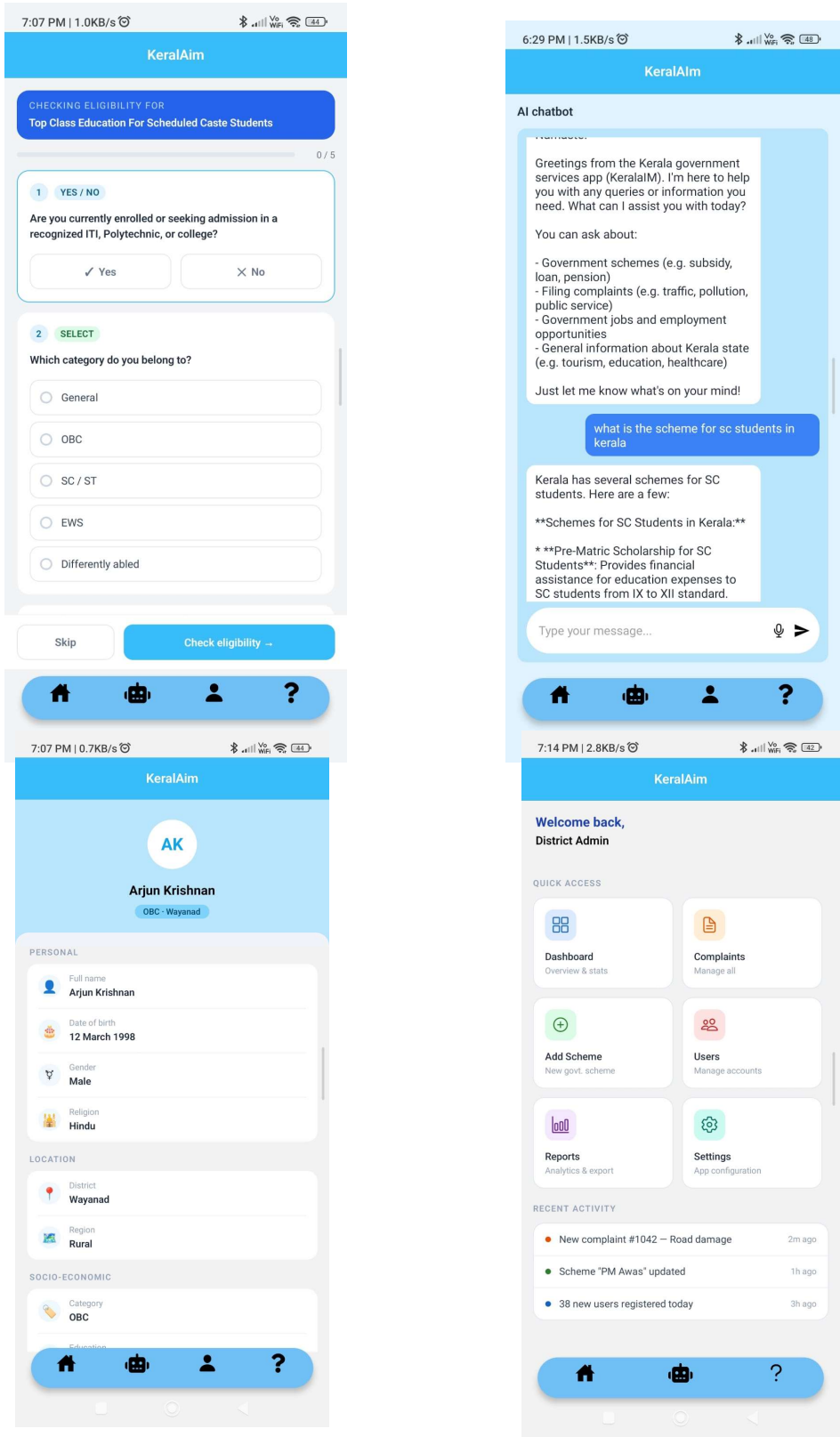
If user asks about:
- schemes → suggest relevant schemes
- complaint → explain how to file
- jobs → suggest government job sources
- general → give helpful info

Always be simple and helpful.
`,
        },
        {
          role: "user",
          content: message,
        },
      ],
    });
    console.log("AI Response:", response);
    return response.choices[0]?.message.content;
  } catch (error) {
    console.error("AI Error:", error);
    return "Sorry, something went wrong.";
  }
};
```

c) scheme eligibility score calculation function

```
const computeScore = (eligibilityField: EligibilityField, scheme: Scheme, age: number | null): number => {  
  let score = 0;  
  const p = eligibilityField  
  
  const annualIncome = parseInt(p.income || "0");  
  const disability = p.disability === "yes" ? true : false;  
  const parsed = scheme.schemeDetails?.eligibility?.parsed  
  // Hard matching  
  if (parsed?.incomelimit && annualIncome > parsed.incomelimit) return 0;  
  if (parsed?.disability && disability === false && parsed?.disability === true) return 0;  
  if (parsed?.minAge && age !== null && age < parsed.minAge) return 0;  
  if (parsed?.maxAge && age !== null && age > parsed.maxAge) return 0;  
  if (parsed?.gender && p.gender) {  
    const genderList = parsed.gender  
      .filter((g: any) => g !== null)  
      .map((g: string) => g.toLowerCase());  
    if (!genderList.includes(p.gender.toLowerCase())) score += 25;  
  }  
  
  // Soft Matching  
  if (parsed?.caste && p.category) {  
    const casteList = parsed.caste  
      .filter((c: any) => c !== null)  
      .map((c: string) => c.toLowerCase());  
    if (casteList.includes(p.category.toLowerCase())) score += 25;  
  }  
  
  if (parsed?.occupation && p.occupation) {  
    const occupationList = parsed.occupation  
      .filter((o: any) => o !== null)  
      .map((o: string) => o.toLowerCase());  
    if (occupationList.includes(p.occupation.toLowerCase())) score += 35;  
  }  
  
  if (parsed?.incomelimit) {  
    const ratio = 1 - (annualIncome / parsed.incomelimit);  
    score += Math.round(ratio * 10);  
  }  
  console.log(`${scheme.slug} : ${score}`)  
  return Math.min(100, score);  
}
```

9.5 Other Major Output Screen



10. REFERENCES

- **Expo Official Docs** – <https://docs.expo.dev/>
- **Node.js Documentation** – <https://nodejs.org/en/docs>
- **NativeWind CSS Docs** – <https://www.nativewind.dev/v5>
- **React Hook Form Docs** – <https://react-hook-form.com>
- **React Query** – <https://tanstack.com/query/v4/docs/framework/react/installation>
- **MongoDB Inc.** 2025. *Introduction to MongoDB* – <https://www.mongodb.com/docs/manual/introduction/>
- **Auth0.** 2024. *Introduction to JSON Web Tokens (JWT)* – <https://jwt.io/introduction>
- **GeeksforGeeks** - 2025. *Building REST API with Node.js, Express, and MongoDB.* - <https://www.geeksforgeeks.org/>
- **Groq Docs** - <https://console.groq.com/docs/overview>
- **Refresh Token Basics** - <https://oneuptime.com/blog/post/2026-01-15-react-native-jwt-refresh-tokens/view>
- **Integration Testing** – https://dev.to/ali_adeku/guide-to-writing-integration-tests-in-express-js-with-jest-and-supertest-1059
- **Unit Test** – <https://dev.to/dstrekelj/how-to-write-unit-tests-in-javascript-with-jest-2e83>
- **Jest Testing** – <https://www.testim.io/blog/jest-testing-a-helpful-introductory-tutorial/>